

📁 用到的所有文件

我用夸克网盘分享了「EIDE」，点击链接即可保存。打开「夸克APP」，无需下载在线播放视频，畅享原画5倍速，支持电视投屏。

链接: <https://pan.quark.cn/s/cd78c863c8ec>

通过网盘分享的文件: EIDE

链接: <https://pan.baidu.com/s/1Zu-HUVzyWbblBua1S9mPoQ?pwd=tpnm> 提取码: tpnm

--来自百度网盘超级会员v5的分享

<https://www.123684.com/s/BoRRVv-53Cxb> 提取码: UILd

vscode下载 [Visual Studio Code - Code Editing. Redefined](#)

mingw下载 [Releases · niXman/mingw-builds-binaries](#)

NET6下载 [Download .NET 6.0 \(Linux, macOS, and Windows\) | .NET](#)

嵌入式交叉编译器gcc [Arm GNU Toolchain Downloads - Arm Developer](#)

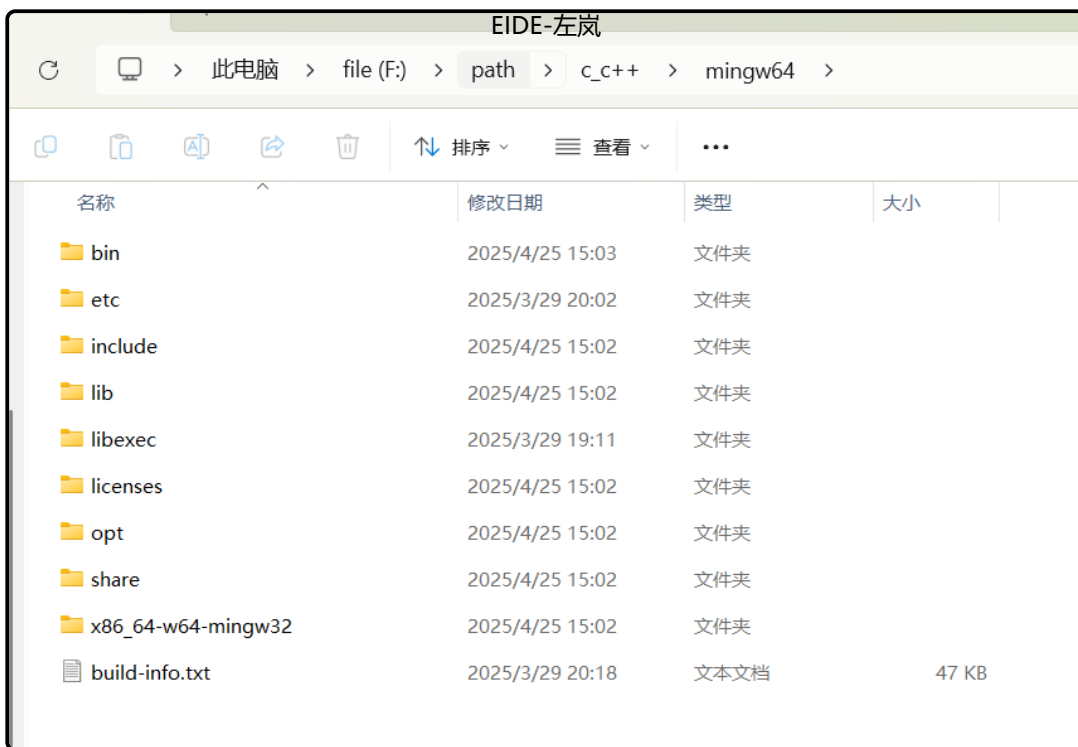
java 环境 下载 [\[Java Archive Downloads - Java SE 8\]](#)
(<https://www.oracle.com/java/technologies/javase/javase8-archive-downloads.html>)

📁 前置操作

首先我们需要配置一下c语言的环境，我们去下载 [Releases · niXman/mingw-builds-binaries](#)，根据自己的情况下载，我是windows，所以我下载win32的版本，后缀我选择UCRT

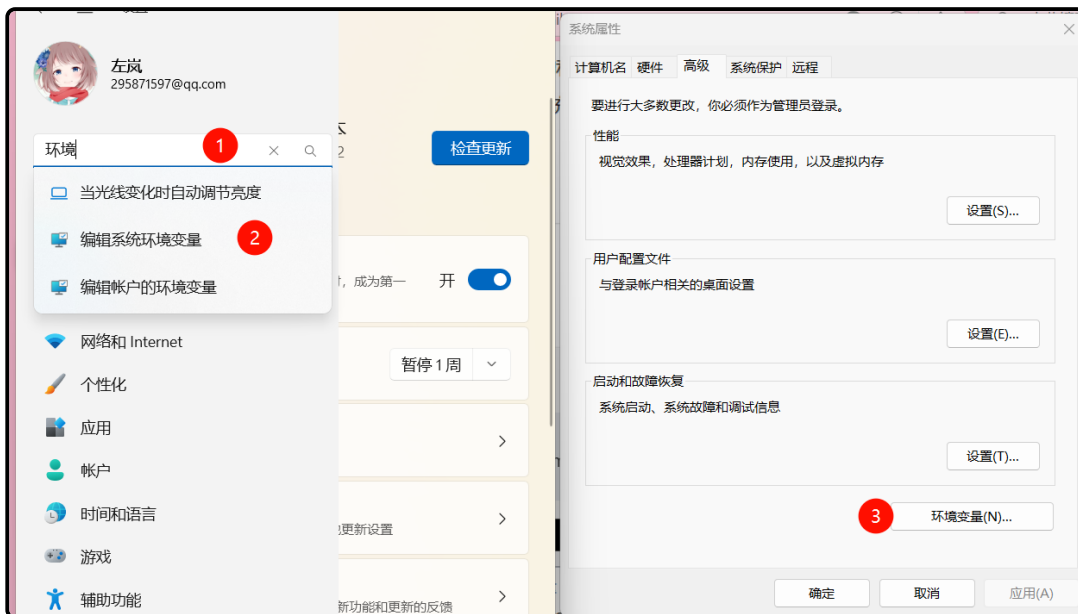
编译选项	全称	解释
MCF	Minimal C Runtime	最小化 C 运行时，通常用于嵌入式系统或对二进制文件大小有严格要求的环境，减少外部依赖。
POSIX	Portable Operating System Interface	遵循 POSIX 标准的编译选项，适用于跨平台开发，尤其在 Unix/Linux 系统上更为常见。
SEH	Structured Exception Handling	Windows 系统中的结构化异常处理机制，用于捕获和处理运行时异常。
MSVCRT	Microsoft C Runtime Library	微软 C 运行时库，Windows 系统上程序依赖的标准 C 函数库，提供文件操作、内存管理等功能。
UCRT	Universal C Runtime	微软的通用 C 运行时库，提供对现代 Windows 系统的支持，替代了 MSVCRT，适用于 Windows 10 及以上版本。

这里注意一下，最好是放在一个自己能找到且不会误删的路径，我放在path文件夹中，后续操作的环境变量也在
这里 `F:\path\c_c++\mingw64`



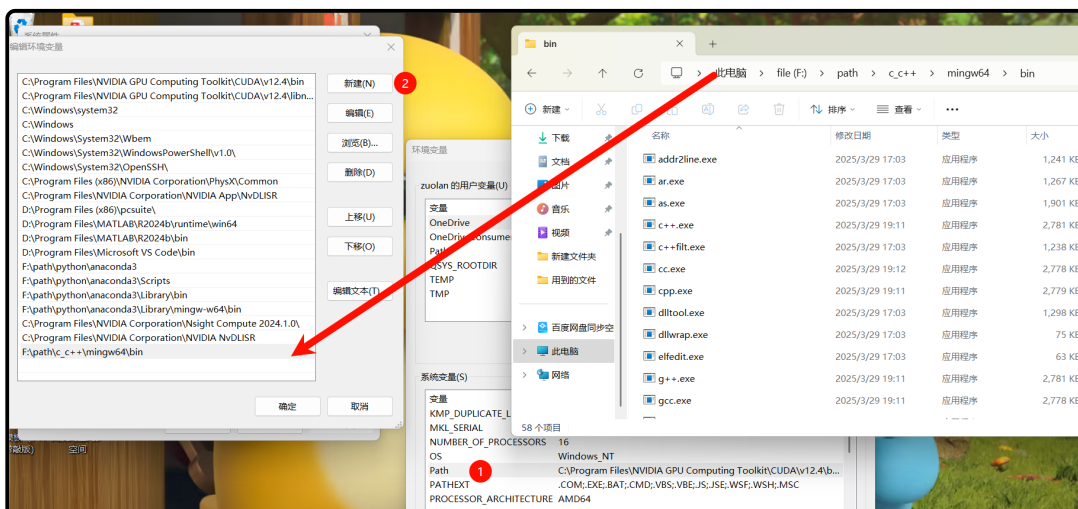
然后我们将这个路径放置到环境变量中

win+i呼出控制中心，选择编辑系统环境变量（注意不要选错了）

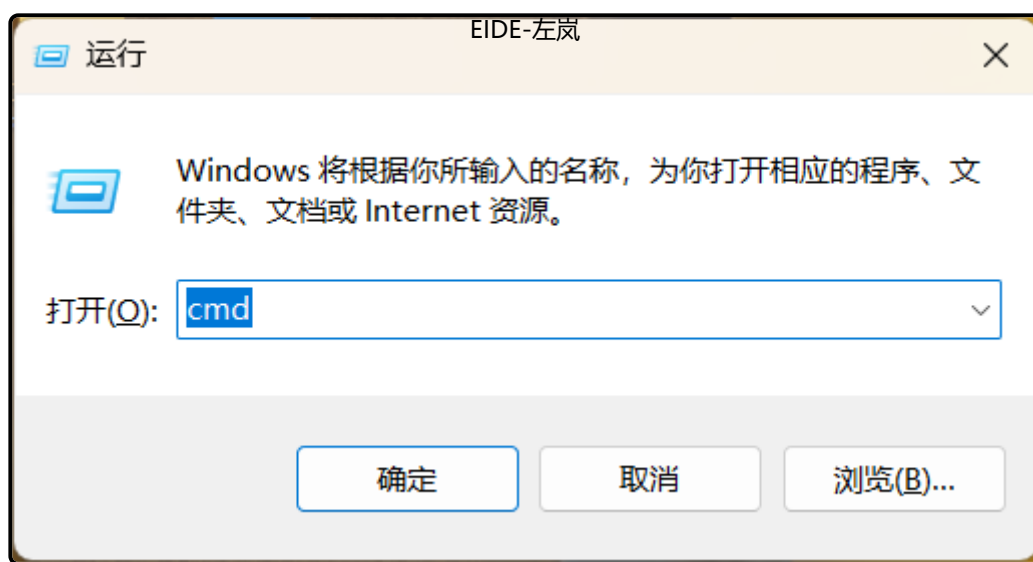


双击【系统变量】中的【path】，下一个界面点击新建，然后把上面配置的路径的bin文件夹路径填进去

F:\path\c_++\mingw64\bin



win+r呼出运行框，输入cmd

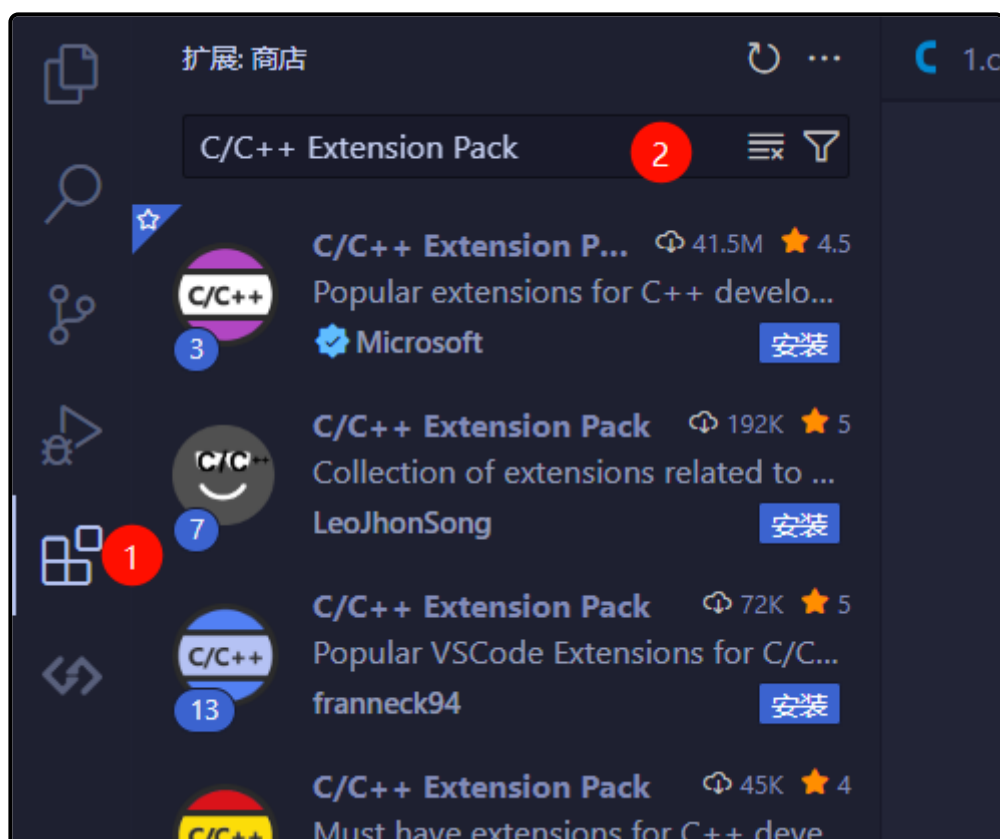


输入 `gcc --version` , 只要有类似输出, 就代表配置完成

```
C:\Users\zuolan>gcc --version
gcc (x86_64-win32-seh-rev2, Built by MinGW-Builds project) 14.2.0
Copyright (C) 2024 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

🔗 C/C++ Extension Pack插件设置

这里安装第一个就行了, 他会帮我们把全部弄好

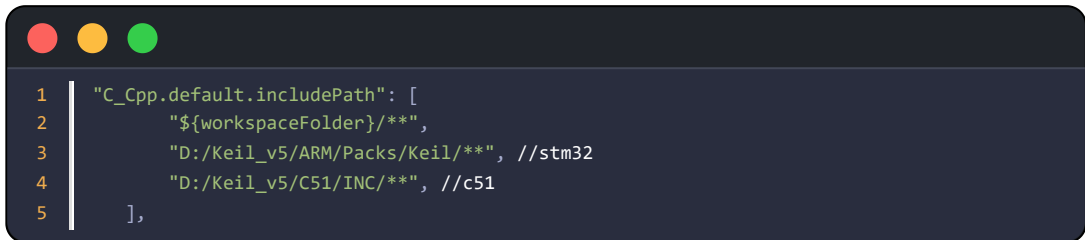


我们在这里我们直接对json进行设置, 点击齿轮后点击文本

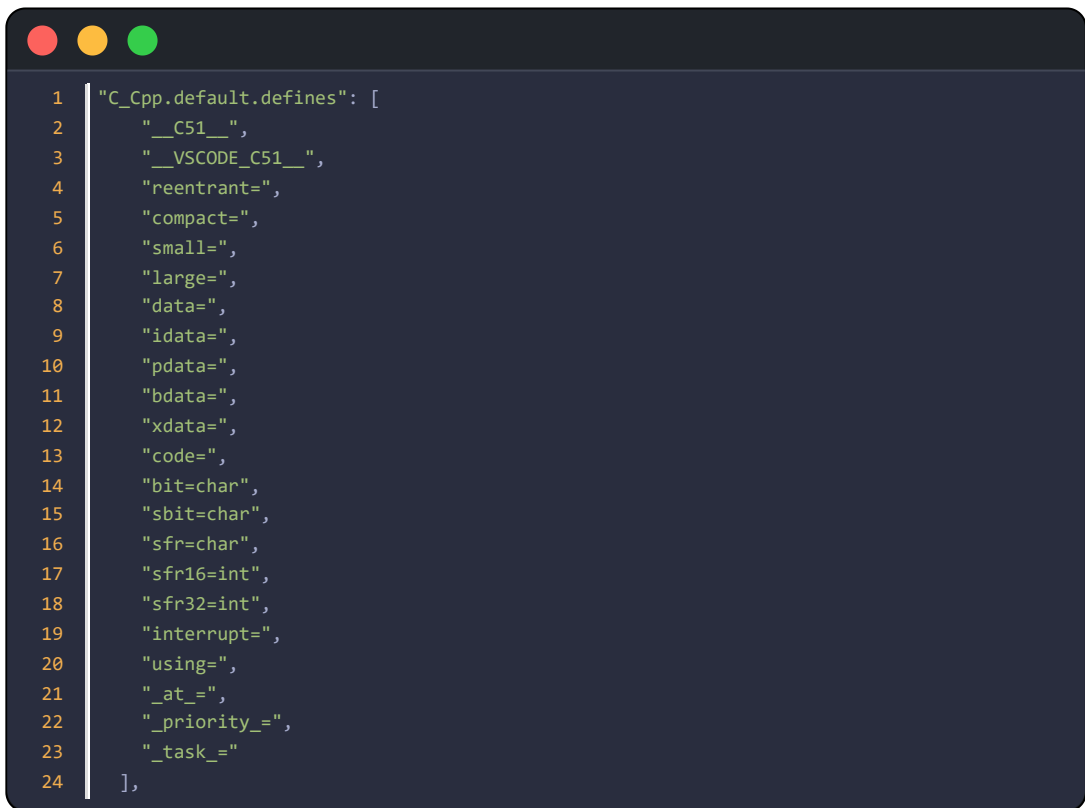


我们开始配置这个json文档，主要是用这个就行了，记得json的分隔符是逗号，也就是如果上一个没有结束标记，那么记得在上一行添加上英文逗号，

此时你的电脑的keil安装了对应的pack，比如f4，f1等，我们直接添加这个路径 `D:\Keil_v5\ARM\Packs\Keil` 就可以，如果你是需要写c51的代码的话，那么就添加 `D:/Keil_v5/C51/INC`，注意一下，这里需要用 `/`，因为 `\` 是转义符号，并且需要在末尾加上 `/**` 来表示所有文件，但是注意，如果添加的越多，那么他扫描的就越慢，所以根据自己的情况自定义一下，这里定义了之后，就不会报错头文件引用相关的了，如果需要写其他板子，比如ti，那么在这里添加一下ti的头文件路径就行了



由于嵌入式的c语言和标准c语言不太一样，所以我们这里配置一下宏定义，防止一些变量没有（主要是c51）

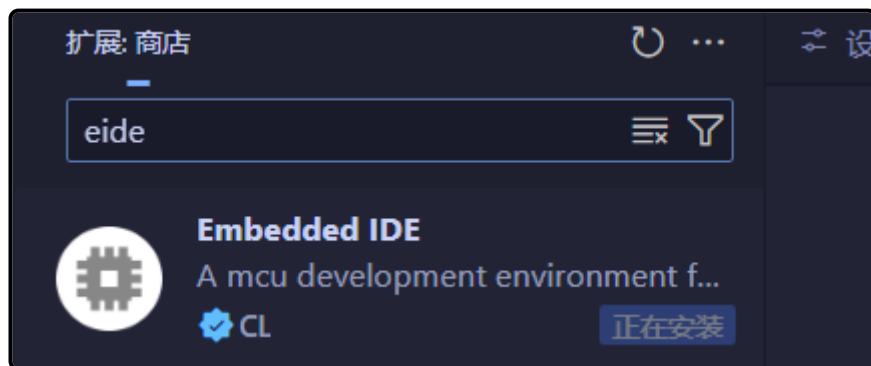


到现在，基本的使用vscode写代码的工作就已经完成了，如果只是使用vscode写代码，keil跑编译和debug的同学，就可以不用继续看了

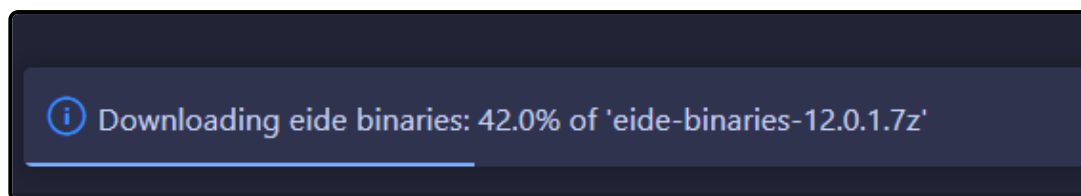
EIDE 插件设置(编译和烧录)

现在就到了最重要的部分，EIDE这个插件可以帮我们进行开发

我们直接搜索EIDE进行安装就行，这是对应的文档 [这是什么？ | Embedded IDE For VSCode](#)



安装好后，他会自动拉取环境依赖（不用魔法上网会比较慢），比如NET6啥的，我也提供一下对应的安装包（因为可能网络问题你会下载失败），在下载完成后，右下角会提示你重启一下vscode

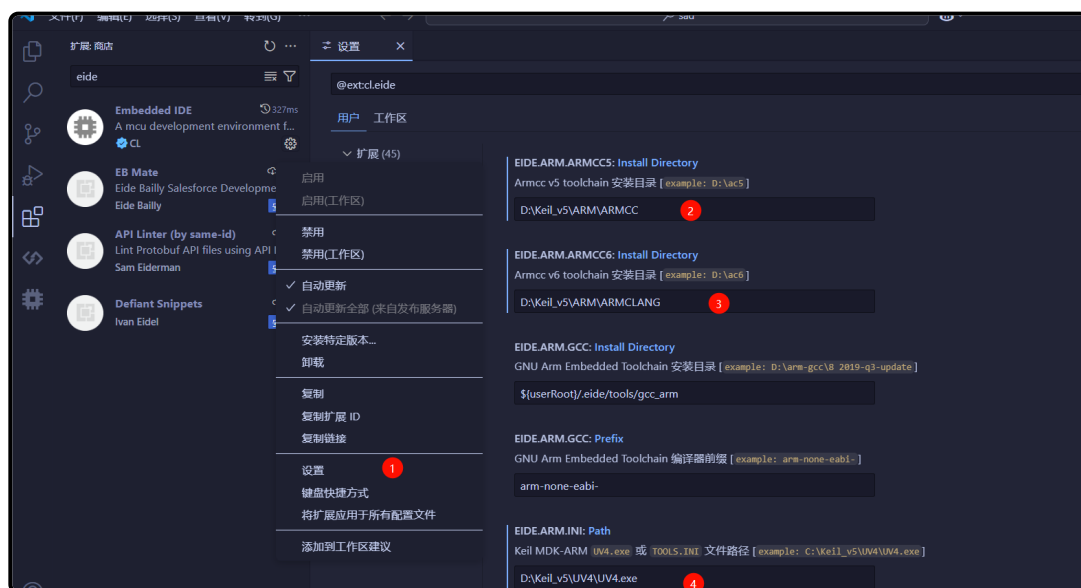


编译器配置

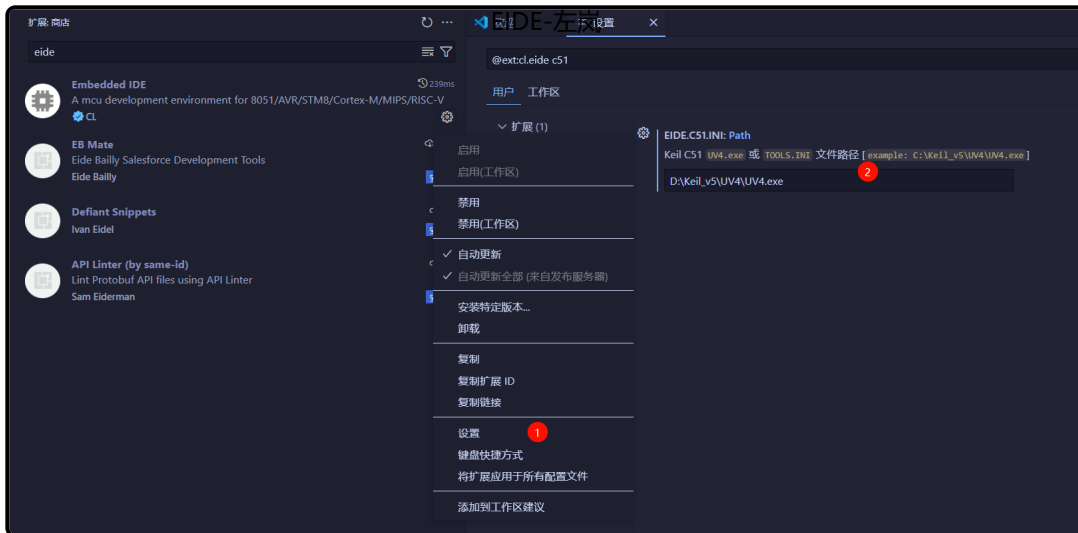
我们可以在这里配置一下相关的编译器（如果你装了keil，那么可以配置一下ac5和ac6（如果有的话），顺手可以配置一下keil的路径）

新版的keil没有自带ac5，所以如果需要用的话可以看我这个视频【【电赛】【STM32】最详细的新版Keil5 c51 mdk安装，ac5编译器安装，STlink驱动解决，pack下载-哔哩哔哩】 <https://b23.tv/WW9oABf>

我这里的路径是 D:\Keil_v5\ARM\ARMCC 和 D:\Keil_v5\ARM\ARMCLANG，D:\Keil_v5\UV4\UV4.exe



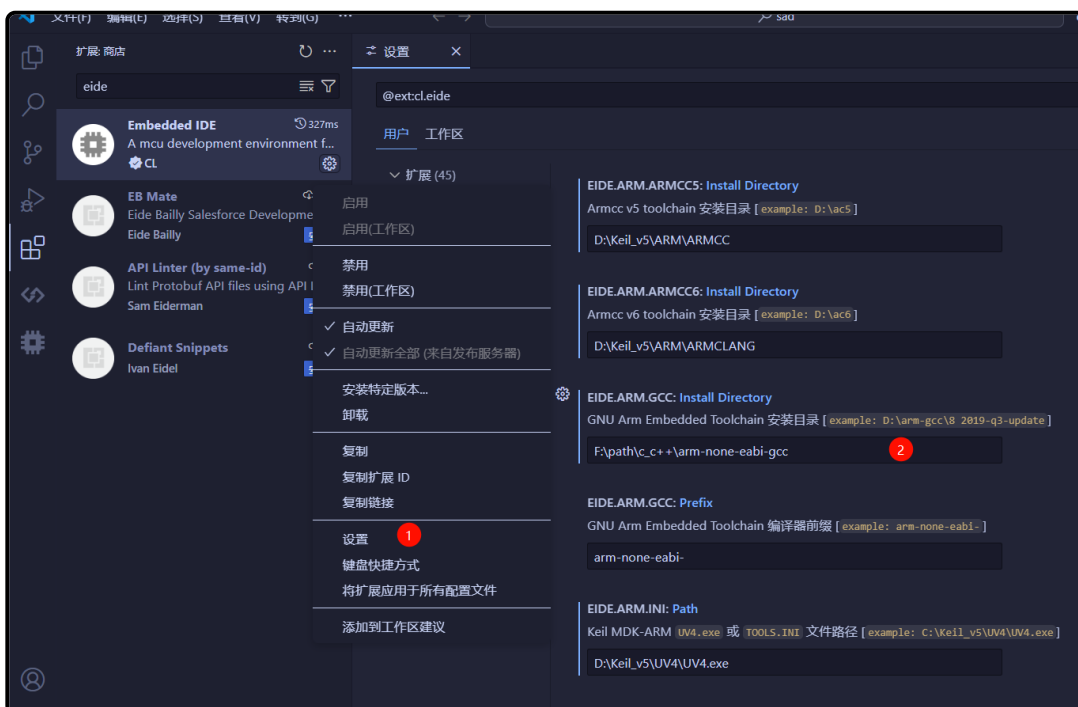
如果你需要用这玩意来编译c51，那么你需要把c51的路径也配置一下



如果你需要使用makefile的方式进行编译，那么需要装一个gcc的编译器 [Arm GNU Toolchain Downloads - Arm Developer](#)，根据自己的情况来装，我是windows，我选择最新的（这个视频录制的时候最新）[arm-gnu-toolchain-14.2.rel1-mingw-w64-i686-arm-none-eabi.exe](#)，还是把他装在我们的path文件夹里面（我方便管理），文件夹为 `F:\path\c_c++\arm-none-eabi-gcc`，然后添加进环境变量就行



随后修改gcc的路径



如果用的是其他的内核，比如ch32这种RISC-V内核的，直接在实用工具里面安装RISC-V GCC Toolchain就行了，
No. 6 / 26

其他需要注意的设置

EIDE-左岚

我们勾选上axf转elf方便后续调试

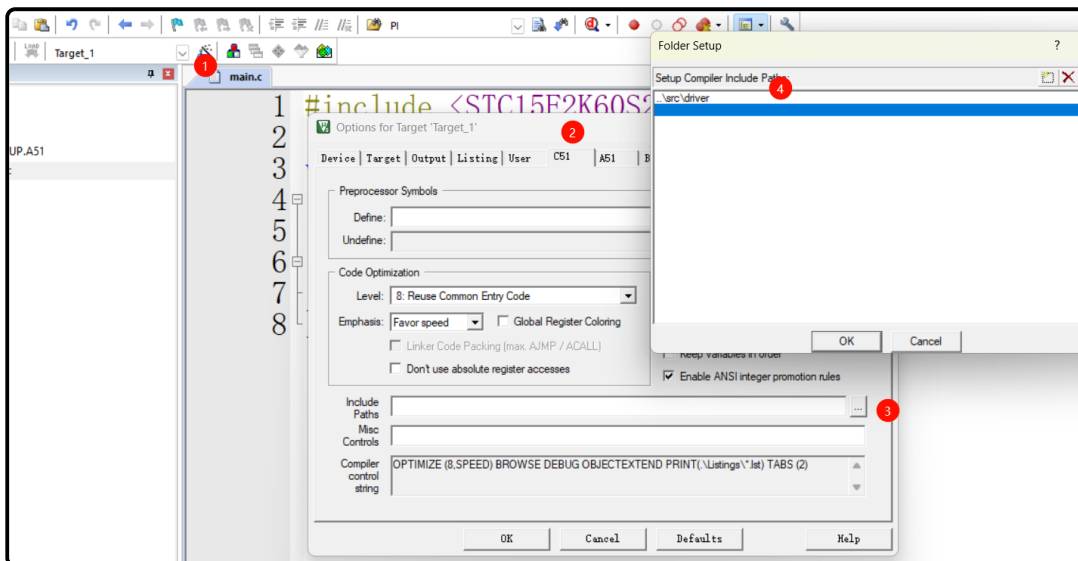
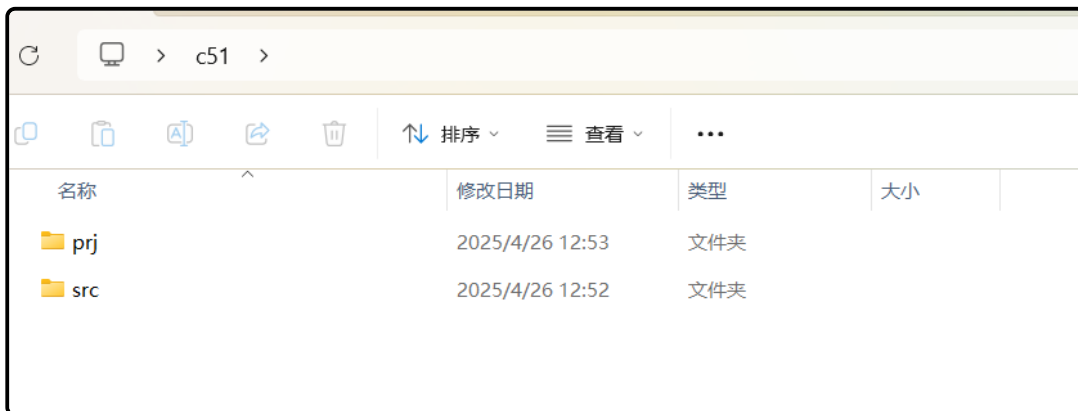
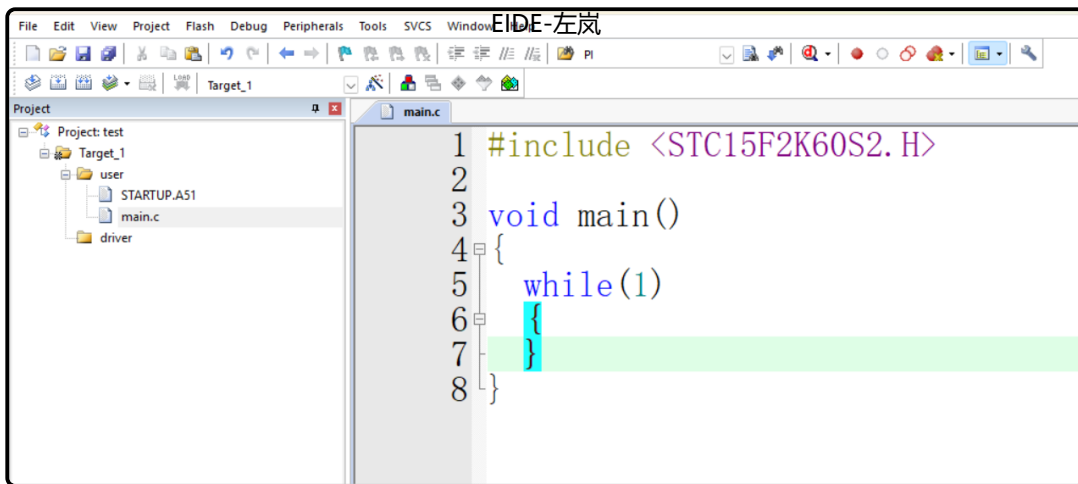


一定要注意我们要配置项目属性和构建器选项（这些在后面都会提及）

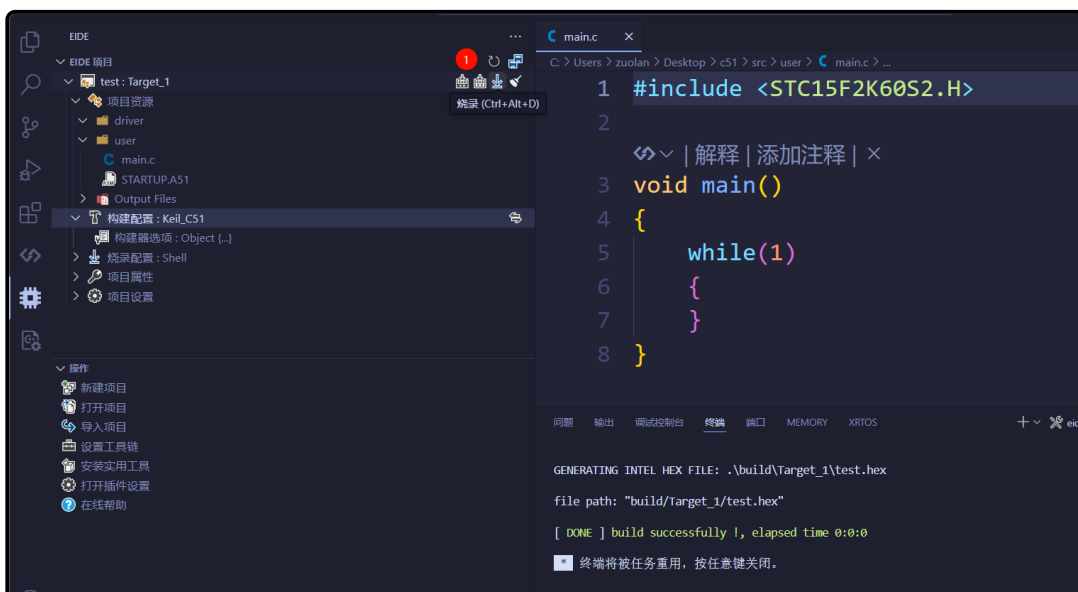


导入keil c51工程（这里不推荐使用eide进行编译烧录，建议只用vscode写代码就行）

这里我不建议使用vscode新建工程，因为可能会造成未知的问题，我们这里直接使用keil新建一个c51的工程（这里就不做示例了），然后随便写一点东西（比如main.c），这里我们规范一点，工程放在prj文件夹里面，代码放在src文件夹里面，其中底层放在driver，应用层放在user，并且不要忘记在魔法棒里面设置一下头文件索引目录



然后我们就直接导入我们的工程就行，选择放在一起，然后我们直接编译就行了

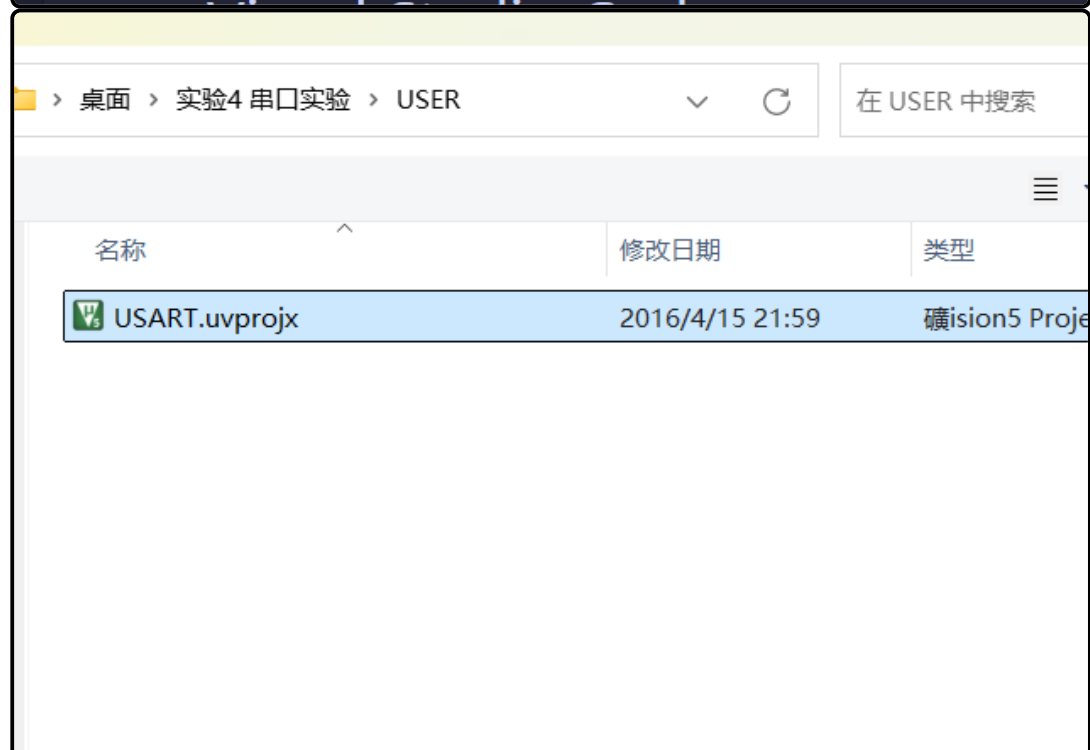
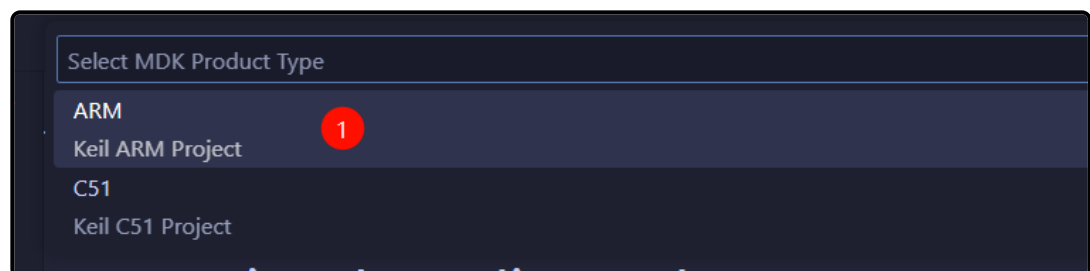
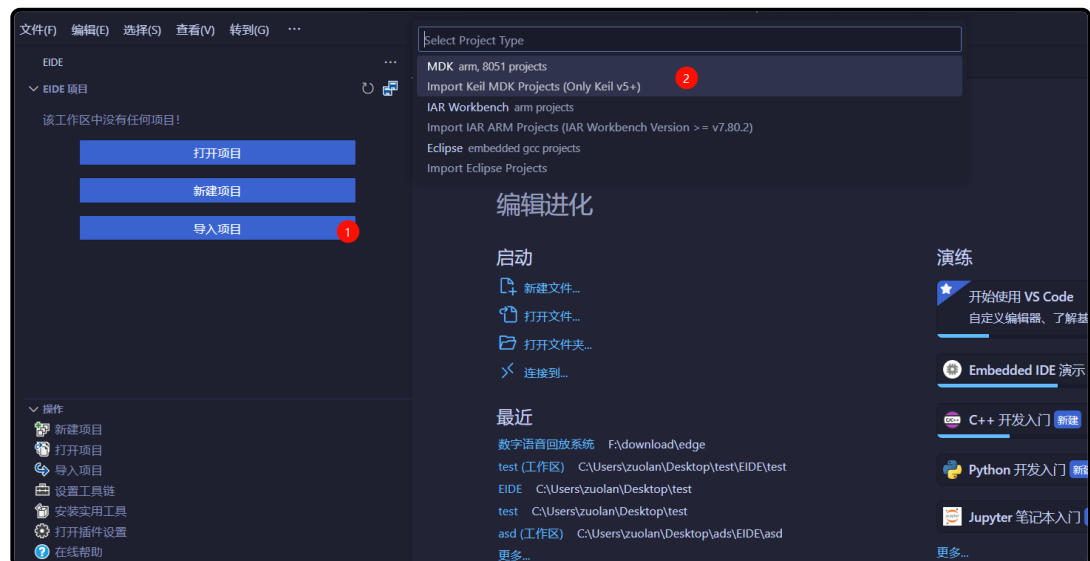


导入keil mdk工程 (这个导入是一次性的, 也就是导入一次之后不会根据新的配置自动修改, 也就是需要重新导入)

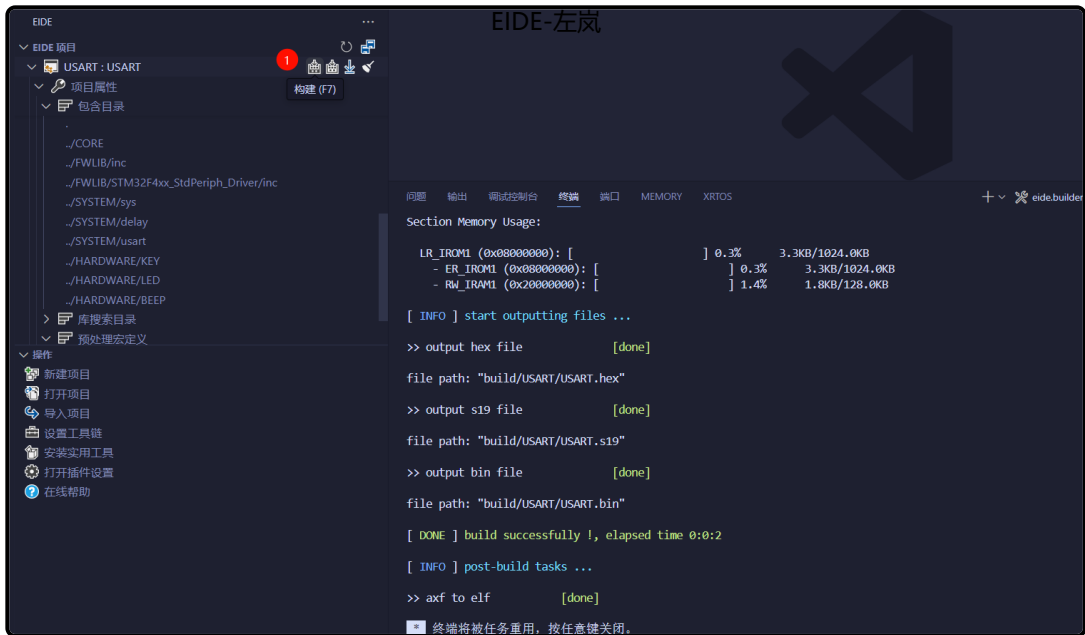
标准库

标准库我就直接拿一个江科大的例子来进行说明

我们直接导入mdk项目



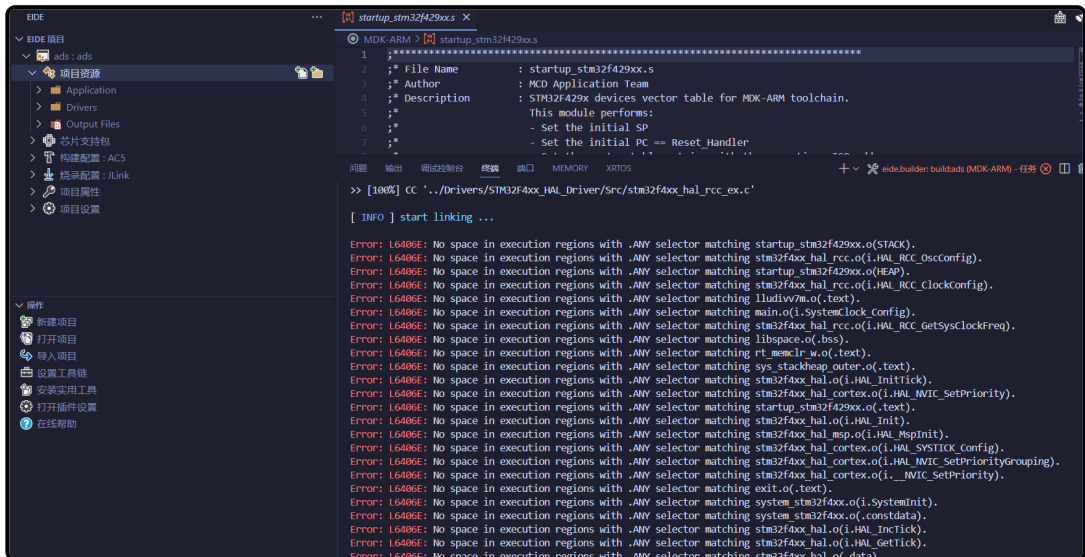
推荐放在同一个目录下, 然后直接编译就行



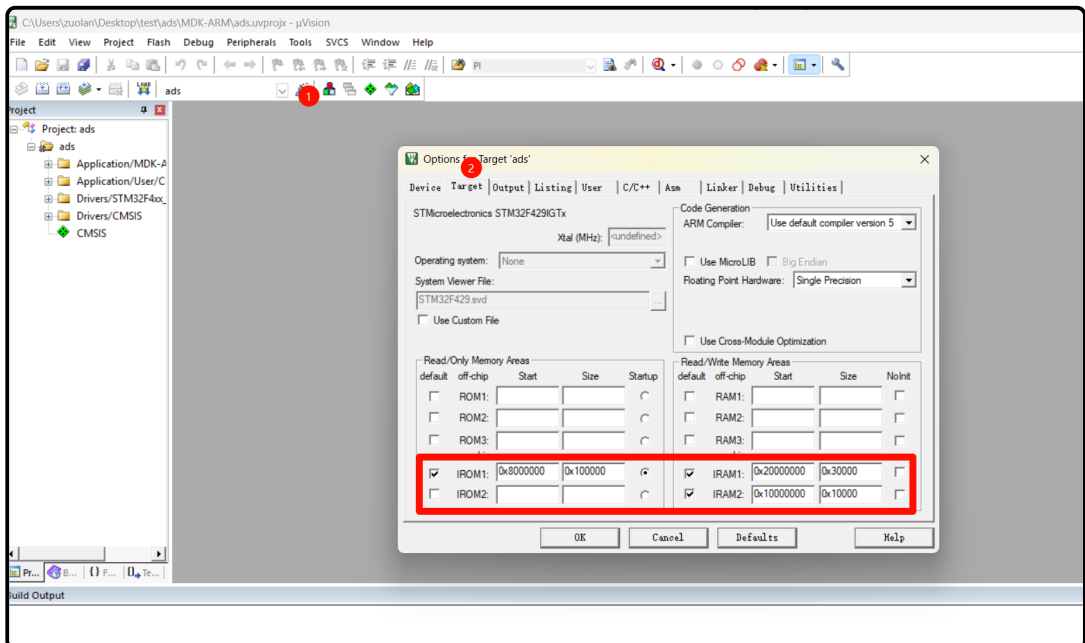
hal库

我这里也直接用cubemx生成，然后我们直接导入对应的工程文件（和上面一样，这里就不截图了）

你会发现编译的时候遇到这个报错，这是因为我们没有分配flash空间导致的，我们打开keil工程

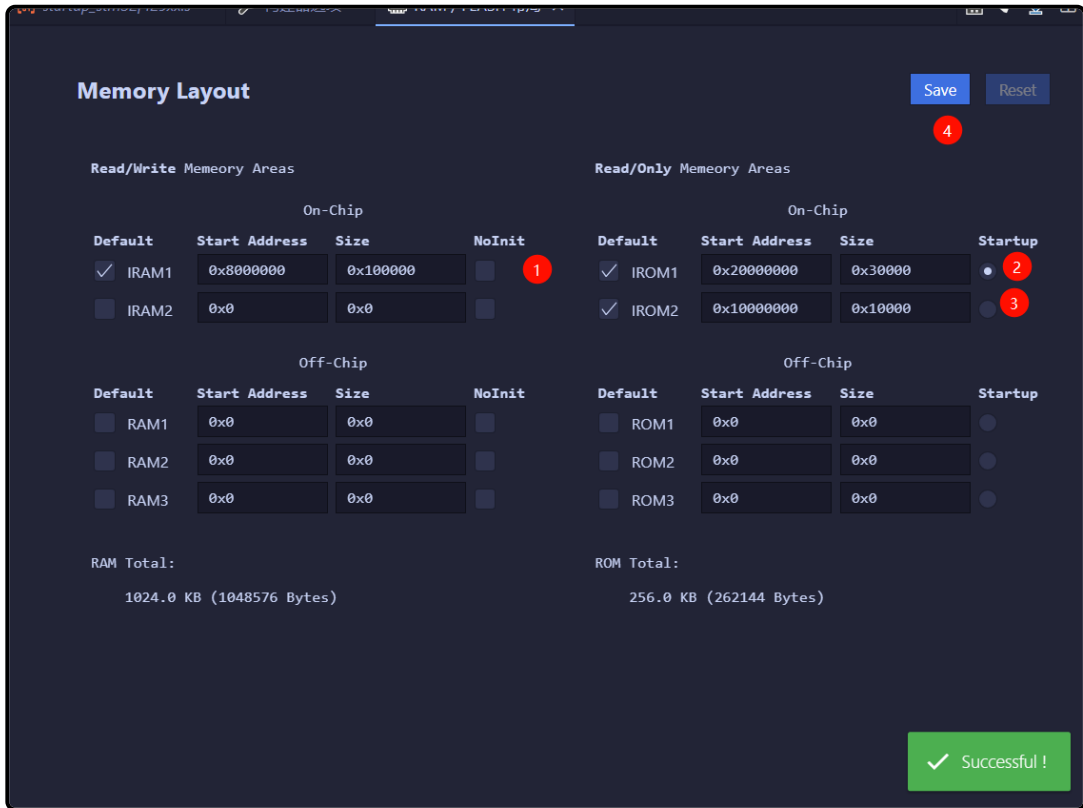


点击魔法棒，在target里面有相关参数，我们将他们记录一下，然后放进eide工程

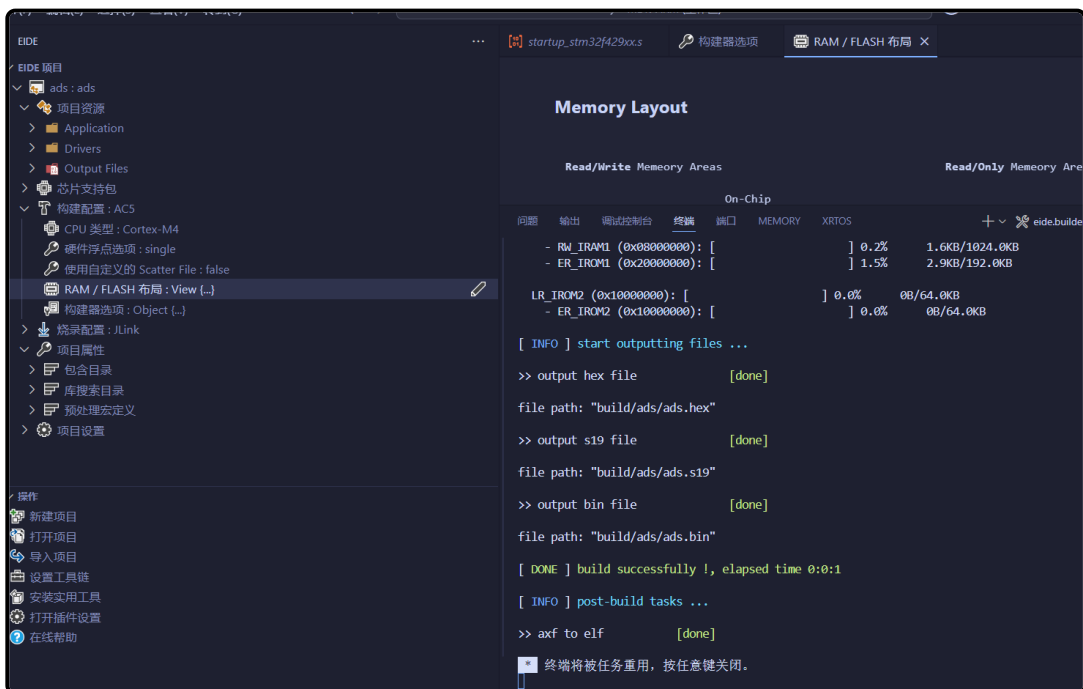




设置好后点击save

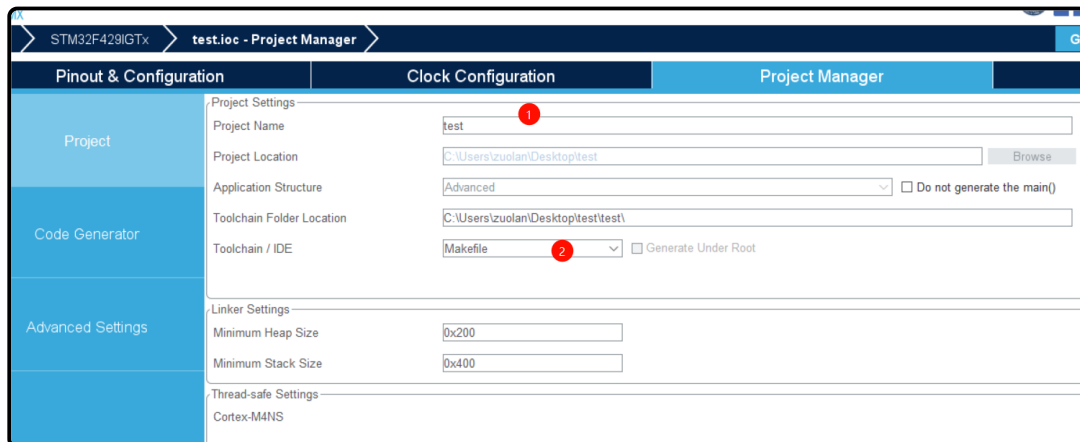


重新编译后就成功了

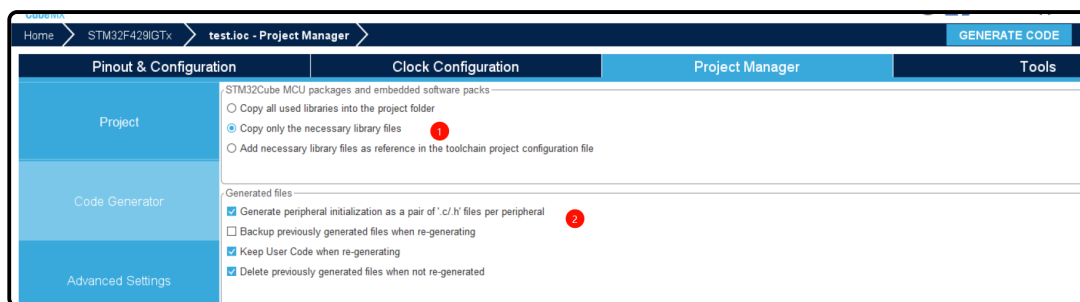


📁 新建Makefile工程

我们在这里使用cubemx生成makefile工程



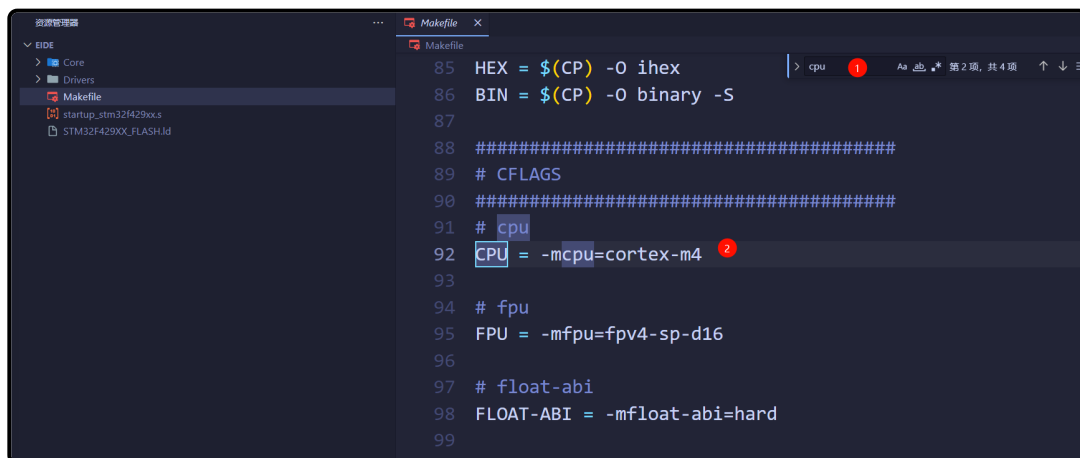
这里注意一下这样选，方便我们看

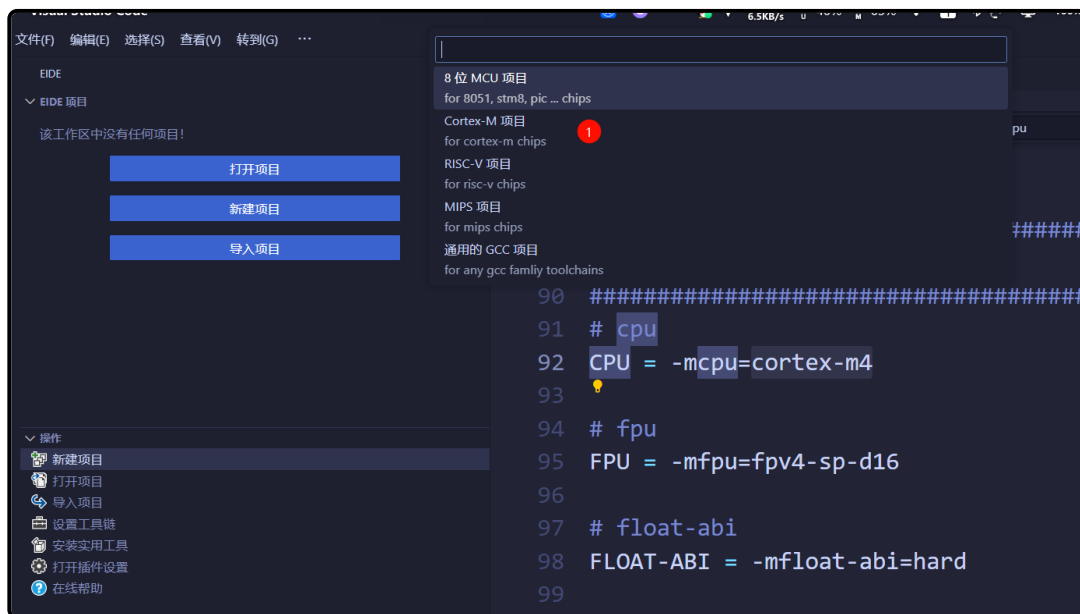
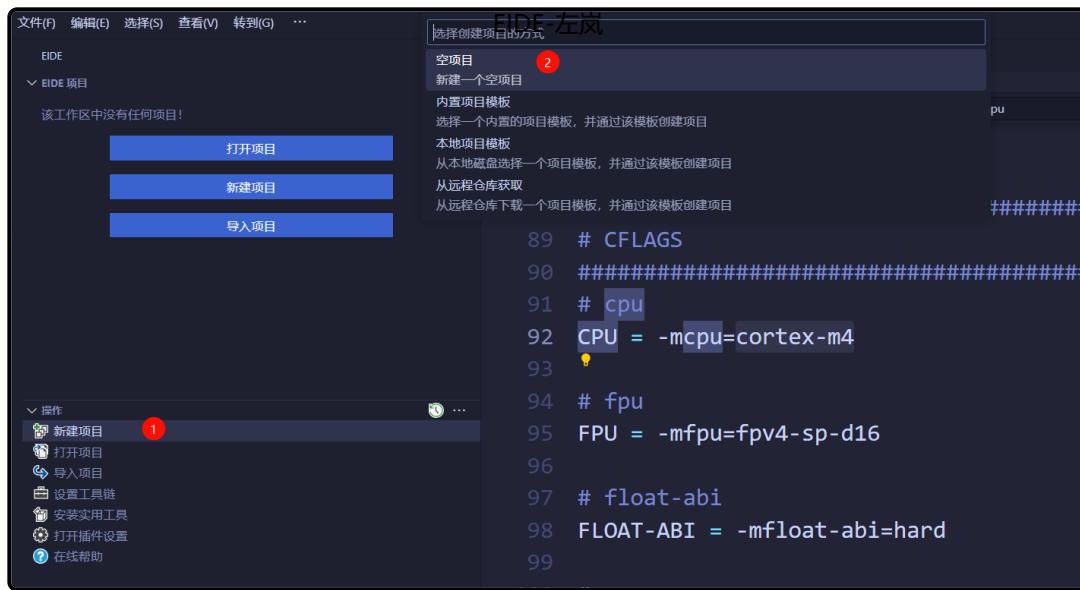


📁 新建EIDE空白工程

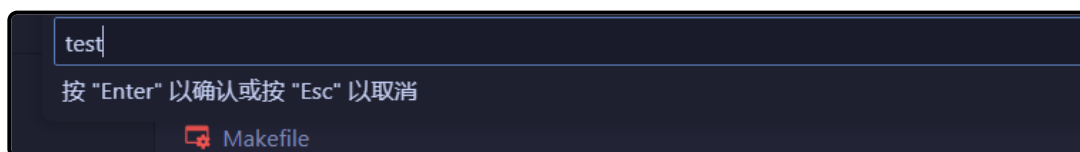
我们新建一个EIDE工程，相关参数我们直接打开makefile看看

首先搜索cpu，然后我们知道这是cortex-m4工程，所以我们新建的时候选择cortex





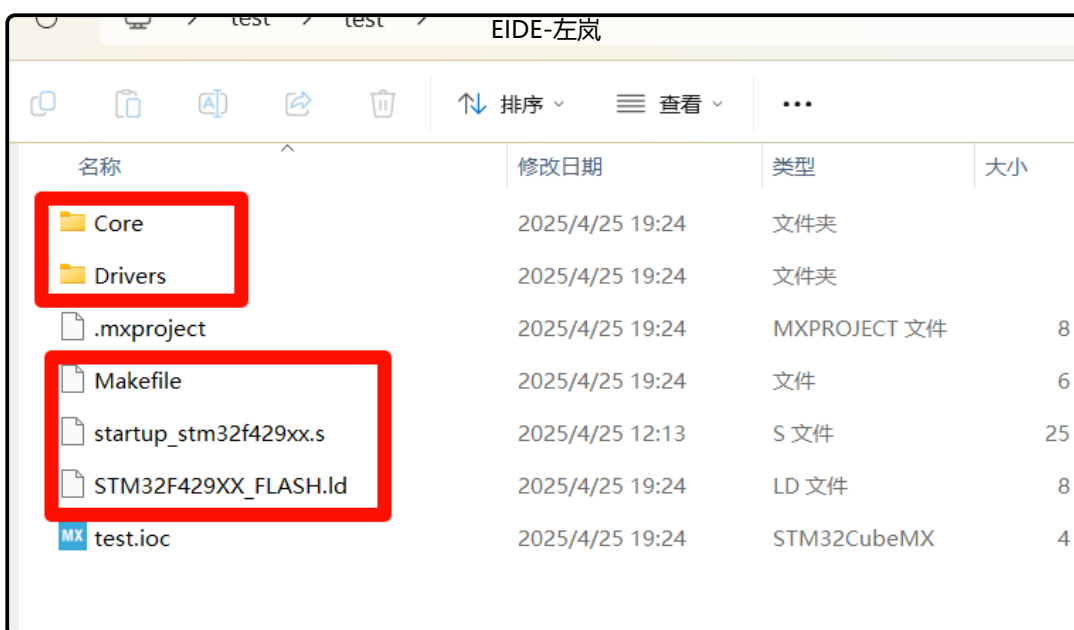
这里的名字要注意一下，和你cubemx新建的工程名字一样



然后根据提示切换工作区

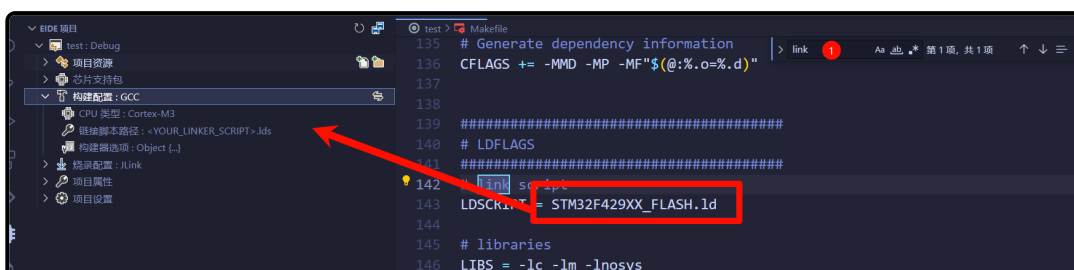


将我们的cubemx生成的代码复制过来（主要是为了避免因为bug导致代码丢失），core和drivers文件夹都要复制过来，还有.s, ld, makefile文件

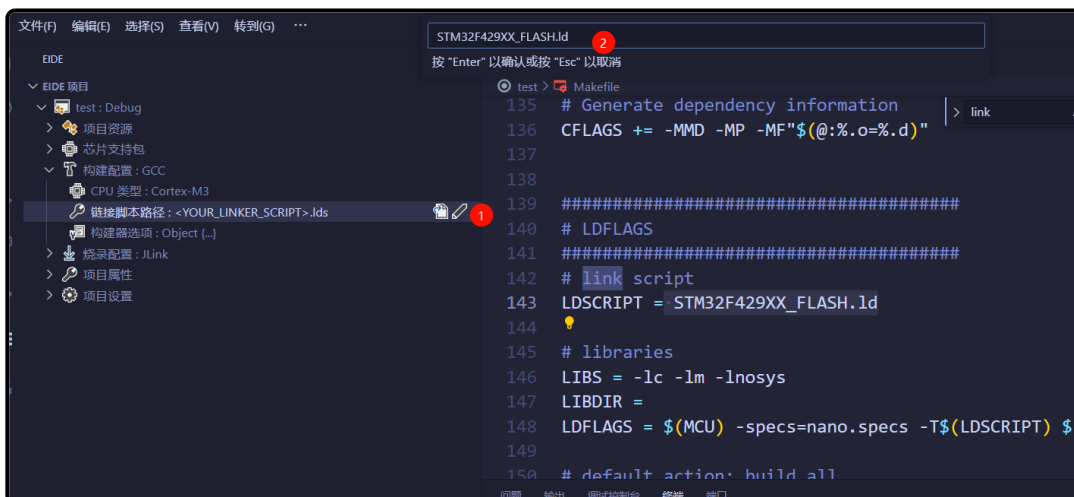


🔗 导入链接脚本与cpu类型

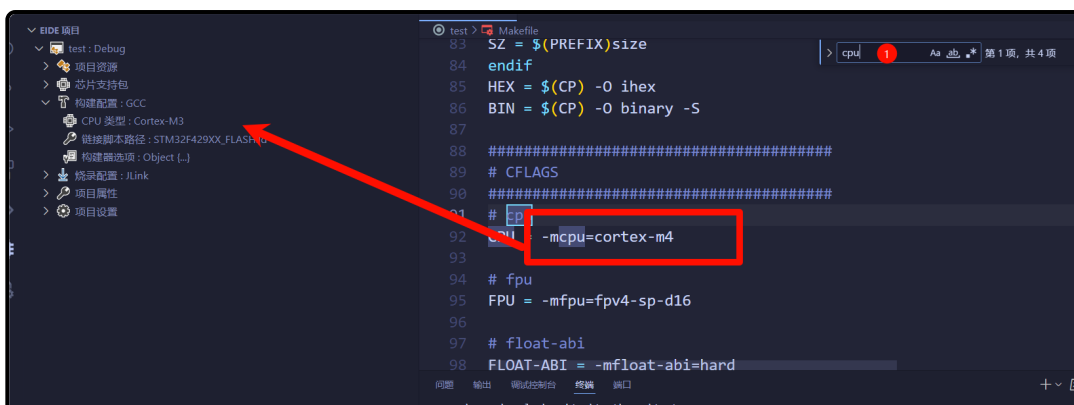
我们再次打开一下makefile文件，查找一下link，也就是我们的连接脚本

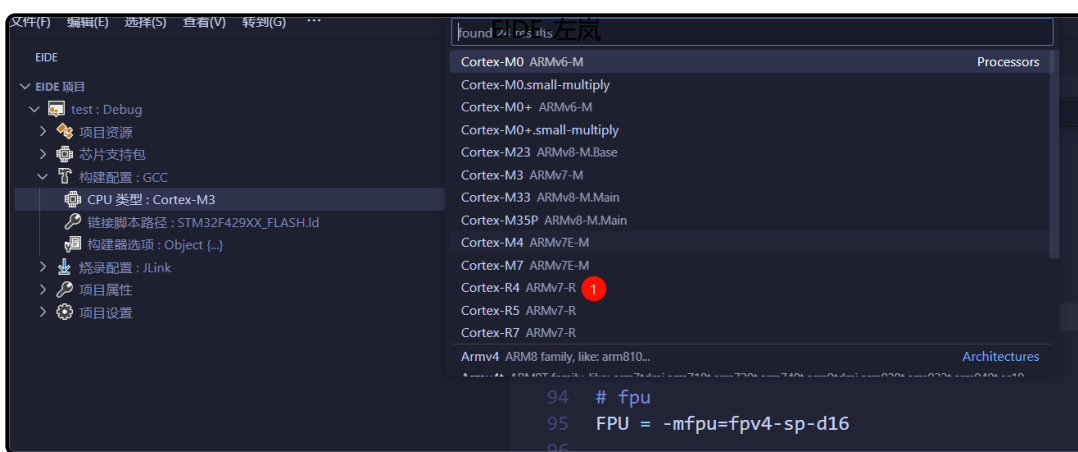


将他的名字复制粘贴过去



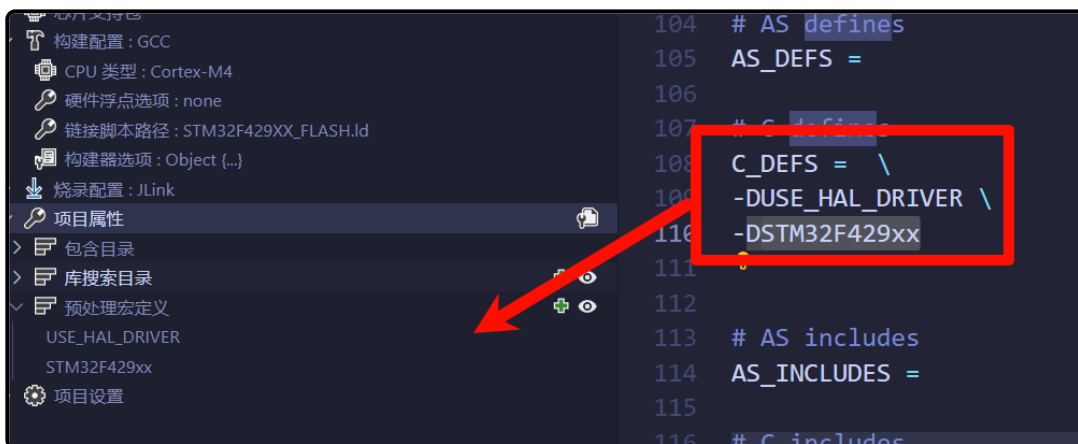
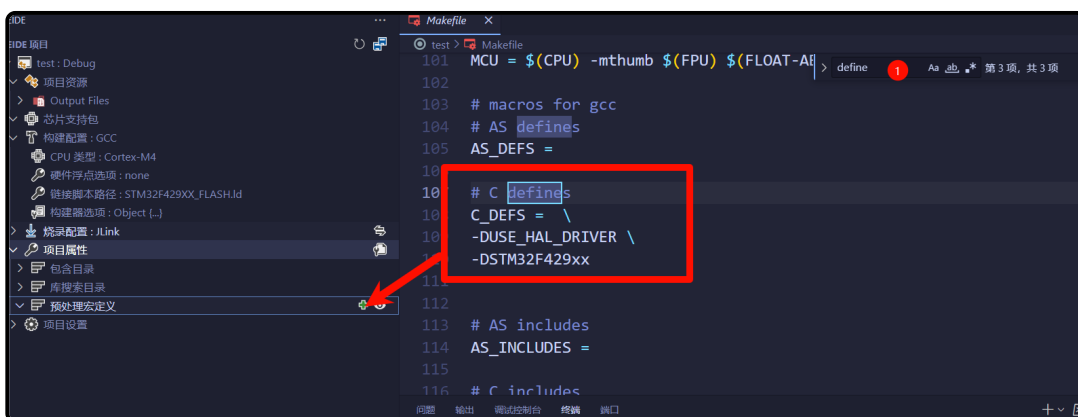
搜索一下cpu，我们发现是cortex M4的，所以我们需要改一下



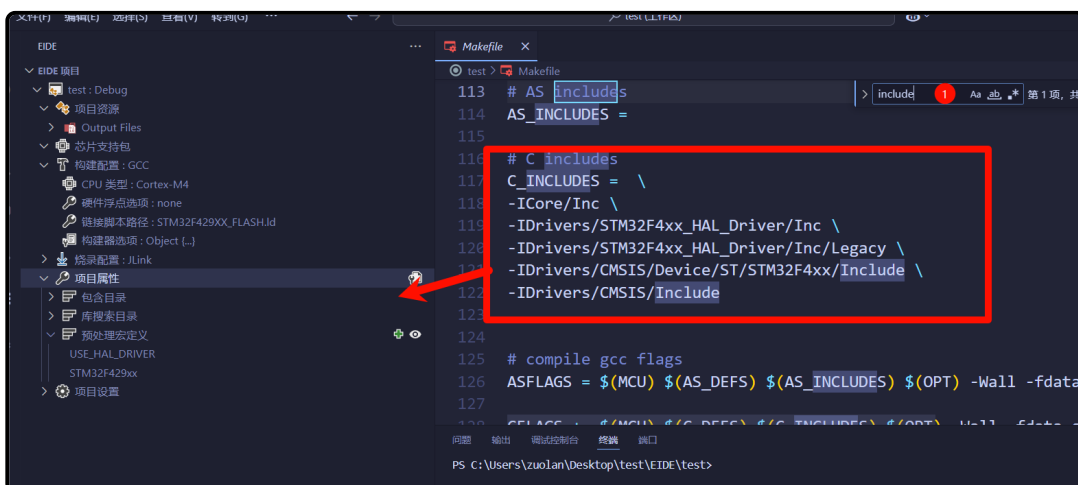


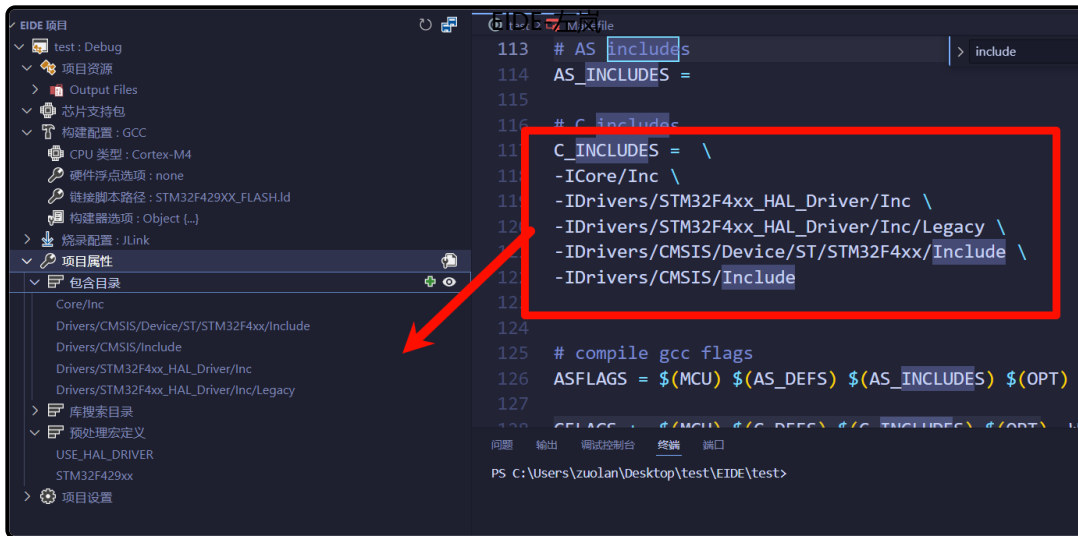
导入cubemx生成的代码与配置

我们首先搜索define，这个是预处理宏定义，我们加进去（去掉D）



接着我们搜索include，导入一下我们的包含目录

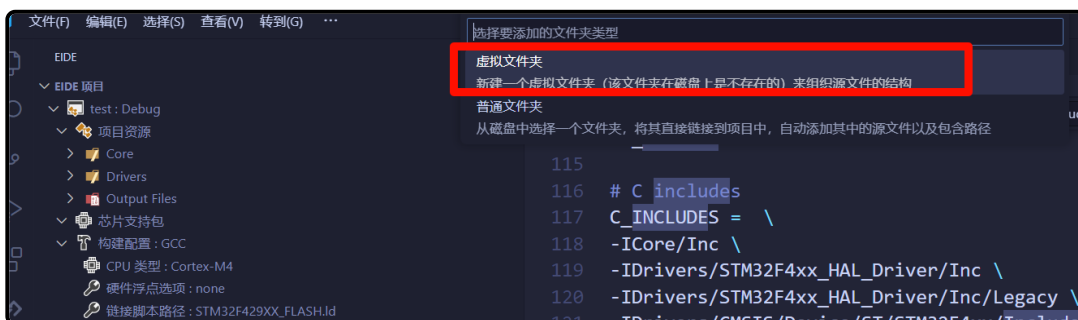




然后将我们的core和drivers文件夹添加进项目资源里面

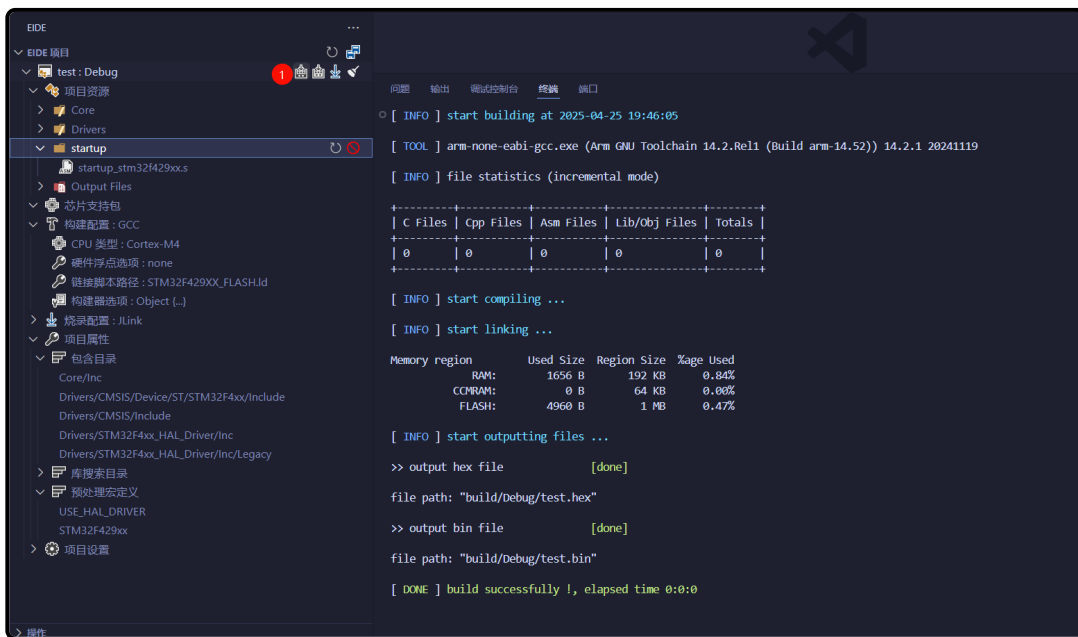


我们新建一个虚拟文件夹放.s启动文件（或者你直接丢进来也可以）



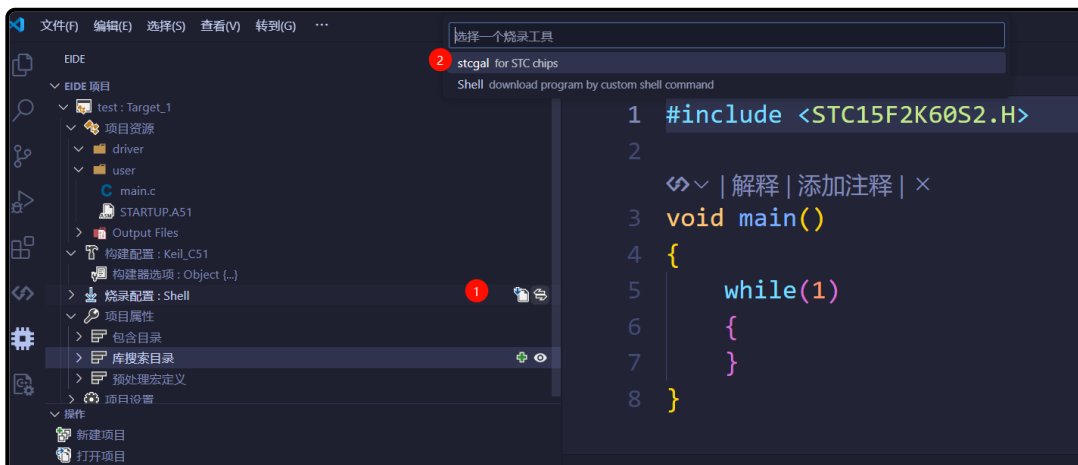


现在所有准备工作做完了，我们可以直接编译，这样就成功了



烧录（C51，这里推荐直接用isp烧录）

我们在这里将烧录配置换成stcgal



接着你需要安装python环境，然后执行一下，对于python的安装，这里就不多赘述了，因为他非常简单，只需要点一下就行



```
PS C:\Users\zuolan\Desktop\c51\prj> pip install stcgal --user
Looking in indexes: https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple
Requirement already satisfied: stcgal in c:\users\zuolan\appdata\roaming\python\python312\site-packages (1.10)
Requirement already satisfied: pyserial>=3.0 in c:\users\zuolan\appdata\roaming\python\python312\site-packages (from stcgal) (3.5)
Requirement already satisfied: tqdm>=4.0.0 in f:\path\python\anaconda3\lib\site-packages (from stcgal) (4.66.5)
Requirement already satisfied: colorama in f:\path\python\anaconda3\lib\site-packages (from tqdm>=4.0.0->stcgal) (0.4.6)
PS C:\Users\zuolan\Desktop\c51\prj>
```

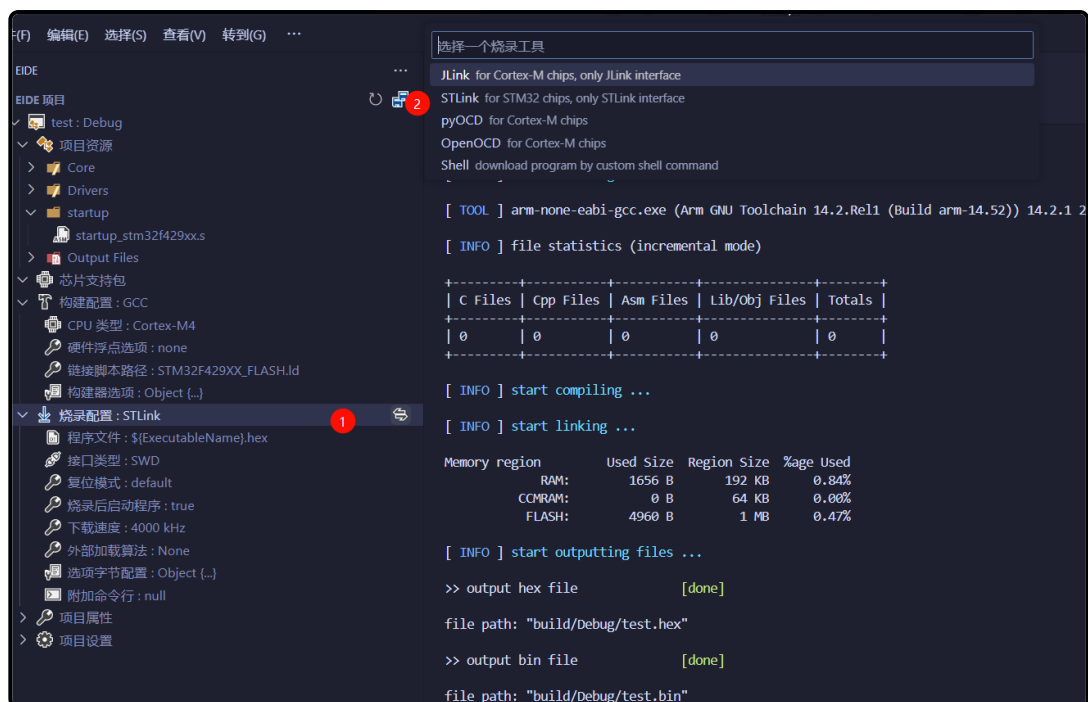
然后我们就可以直接下载了

烧录 (STM32)

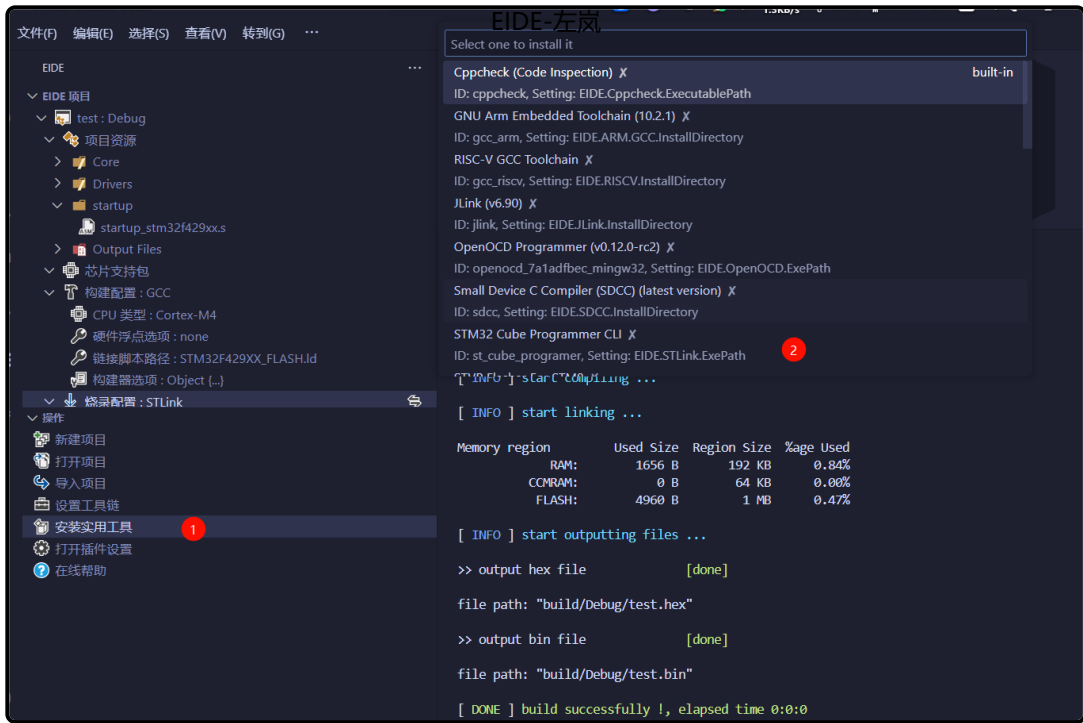
首先确定一下自己手上的烧录器型号，一般来说就三个，stlink，daplink，jlink（这玩意我手上没有，就不测试了，但是官方写了可以）

🔗 stlink

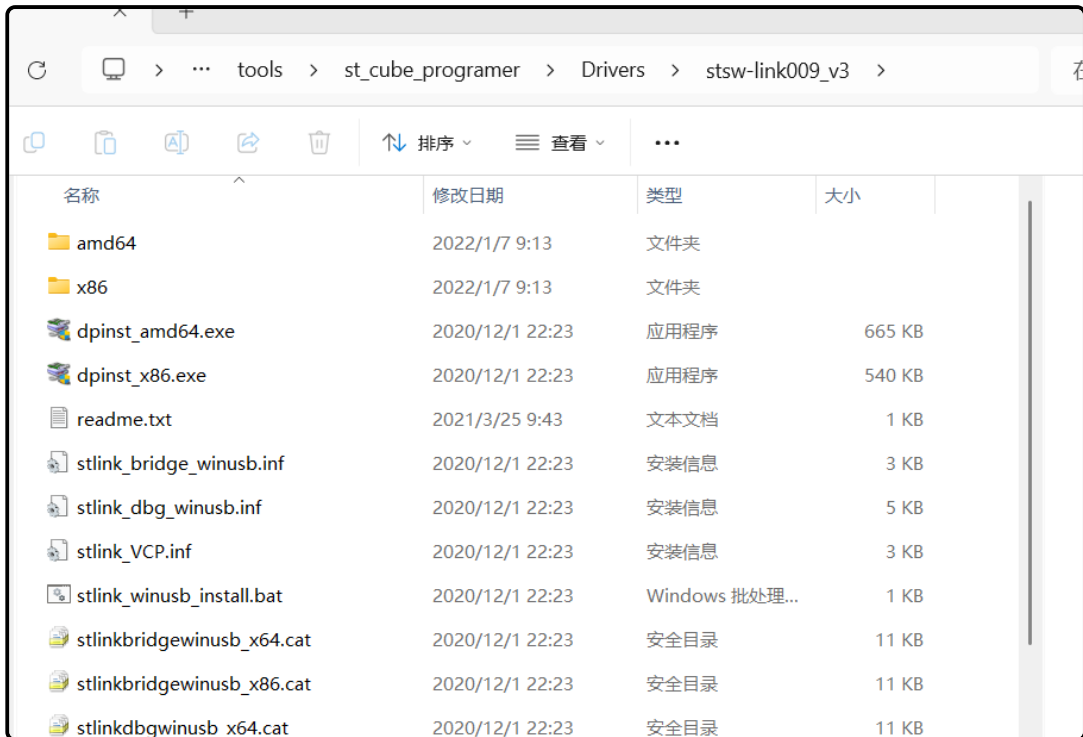
我们直接更换烧录器为stlink就行了



但是在这之前，我们需要装一个stlink的支持STM32 Cube Programmer CLI，我们在实用工具里面可以直接选择，安装好就行了

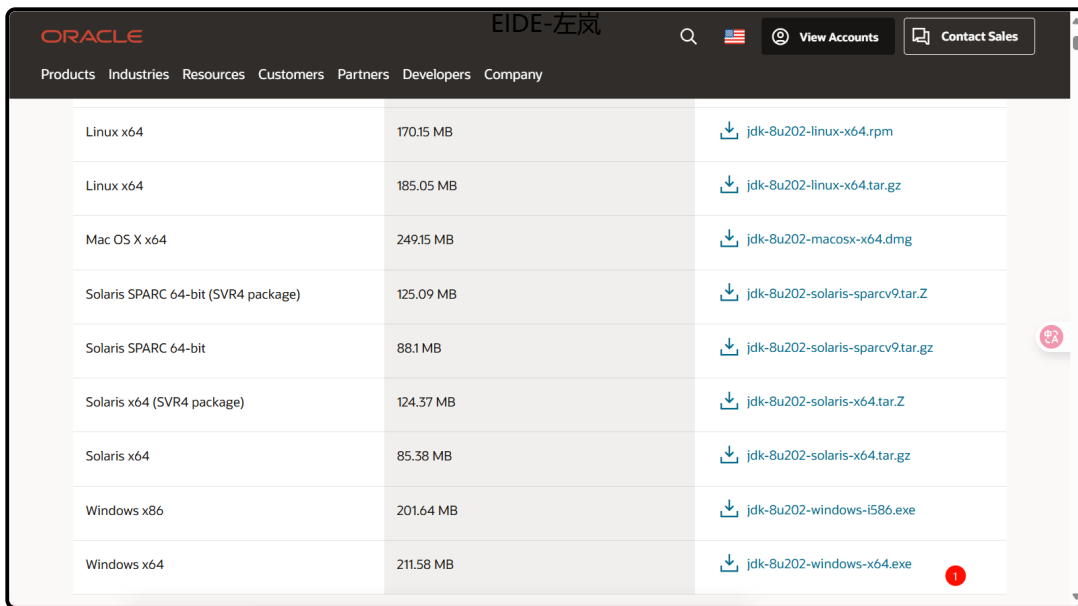


首先确保stlink有驱动，没有的话就去 `C:\Users\zuolan\.eide\tools\st_cube_programmer\Drivers\stsw-link009_v3` 这个地方装一下驱动，双击那个amd64.exe就行了

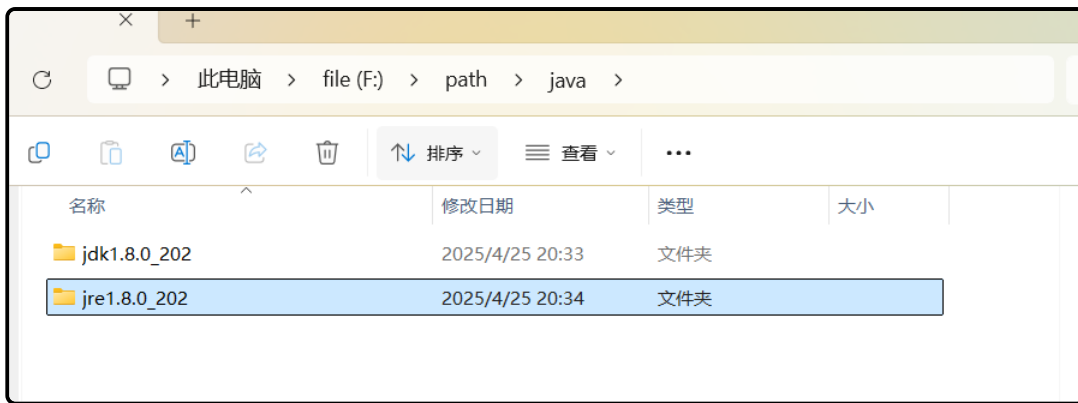


为了保证烧录没问题，我们顺带着在 `C:\Users\zuolan\.eide\tools\st_cube_programmer\Drivers\FirmwareUpgrade` 这里面运行一下那个jar文件（需要有java环境）[Java Archive Downloads - Java SE 8](https://www.oracle.com/java/technologies/javase/javase8-archive-downloads.html) (<https://www.oracle.com/java/technologies/javase/javase8-archive-downloads.html>)

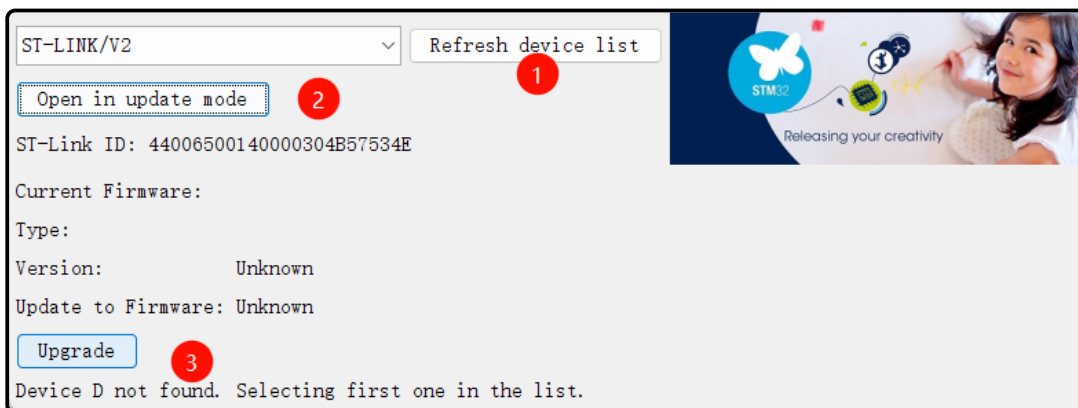
我们下载这个



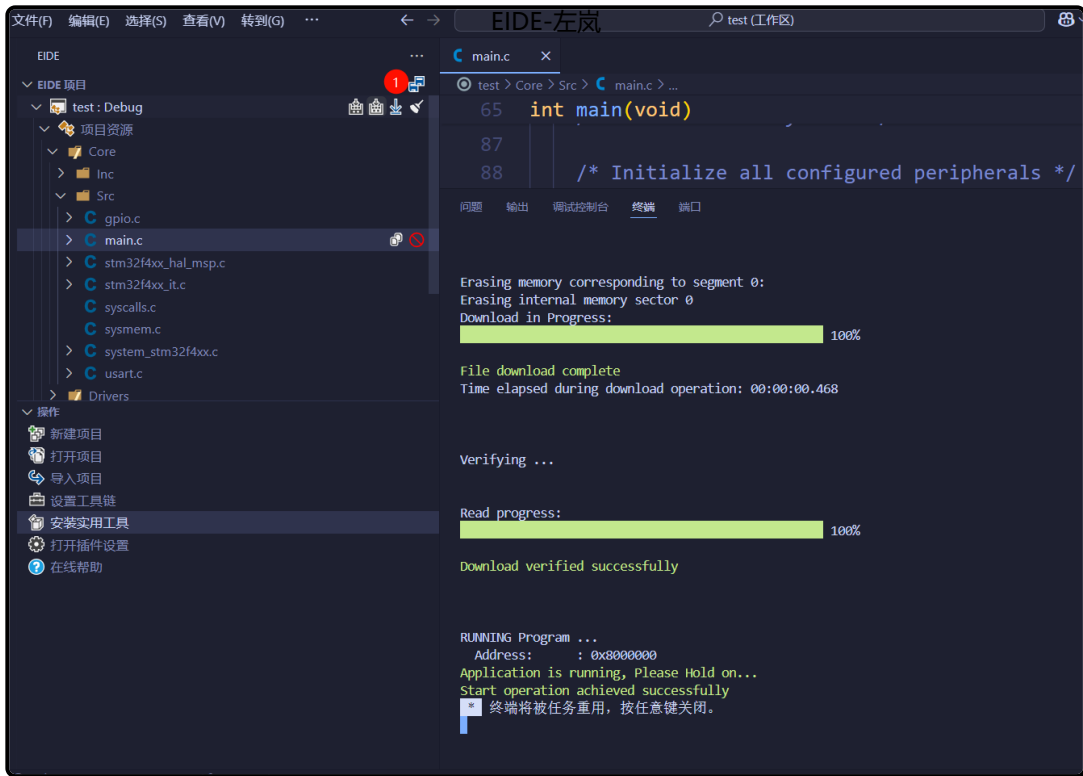
安装的时候还是一样，我们装在path里面 `F:\path\java\jdk1.8.0_202`，注意，装完jdk还有jre，我们装在 `F:\path\java\jre1.8.0_202`



这里不需要配置环境变量，我们直接去 `C:\Users\zuolan\.eide\tools\st_cube_programer\Drivers\FirmwareUpgrade` 运行一下那个 `STLinkUpgrade.jar` 就行（直接双击然后选择java打开就好了），打开后对stlink重新插拔一下，然后refresh刷新后，open in update mode，最后点击upgrade就行，没有扫描到就多次插拔

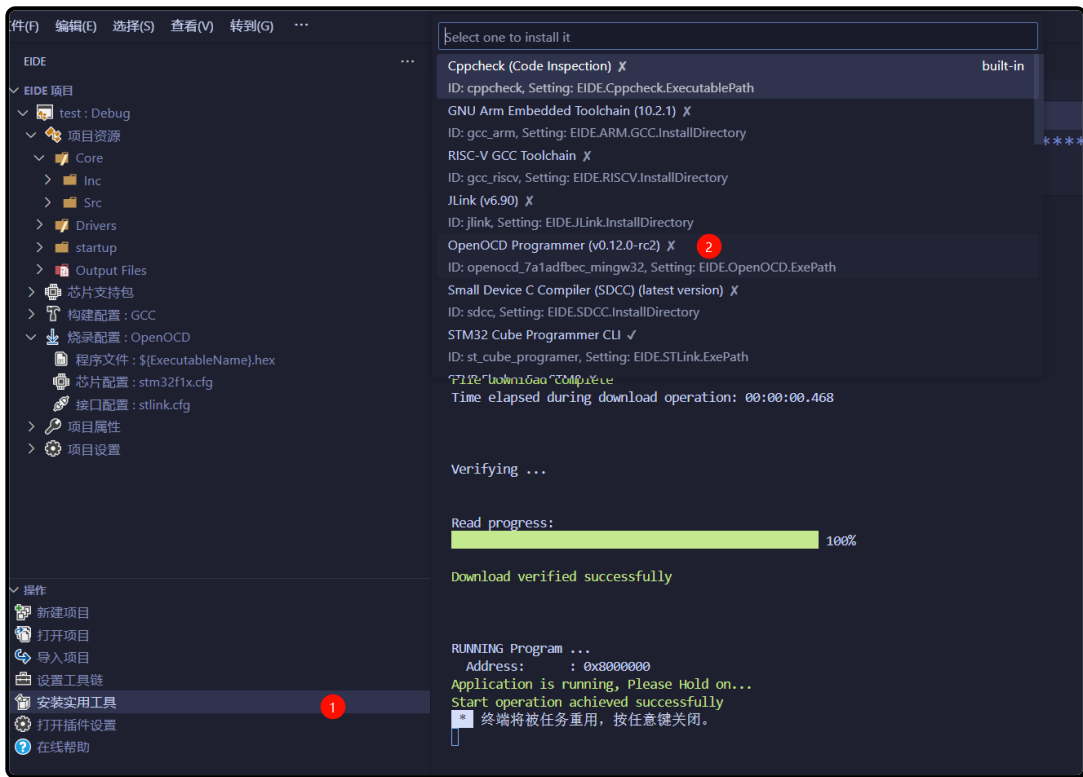


这样就下载成功了

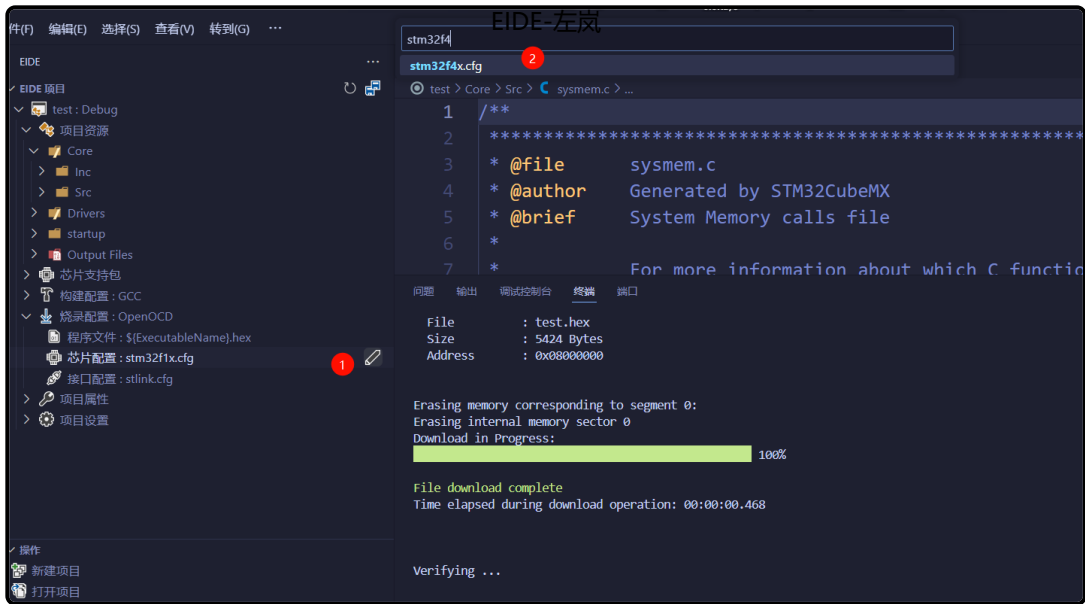


daplink

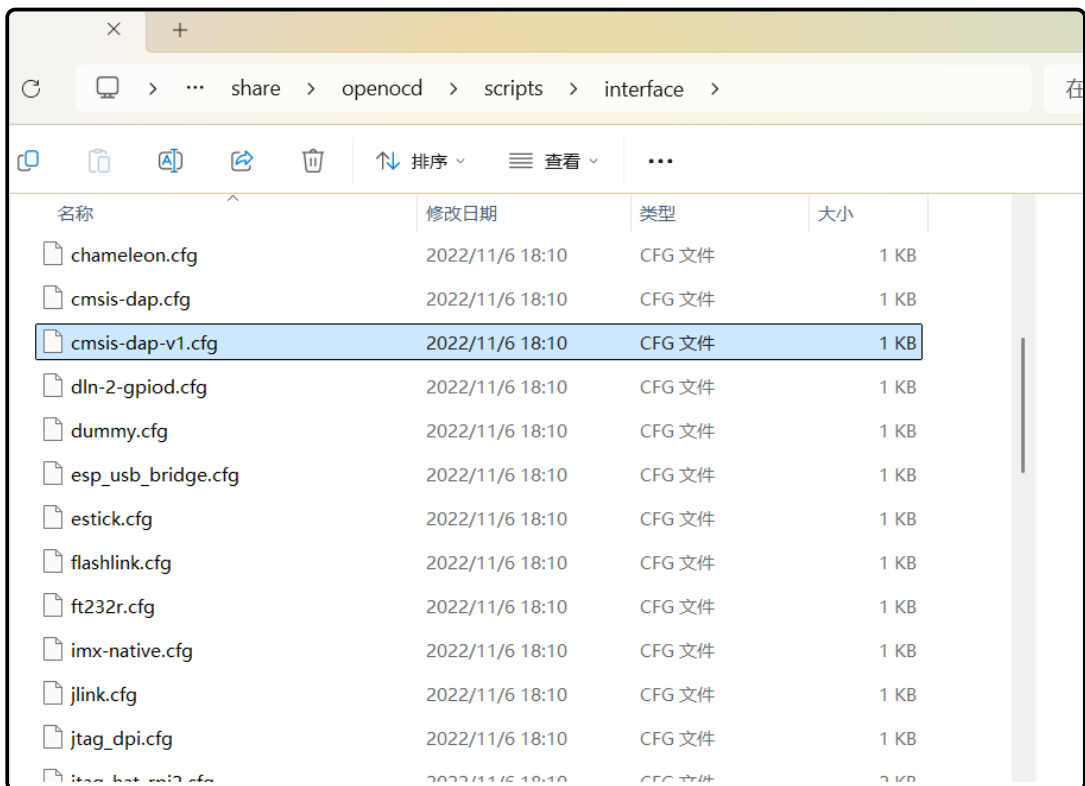
对于daplink, EIDE并没有原生支持,但是我们可以通过OpenOCD来实现烧录,我们首先还是在实用工具里面下载一下openocd programmer



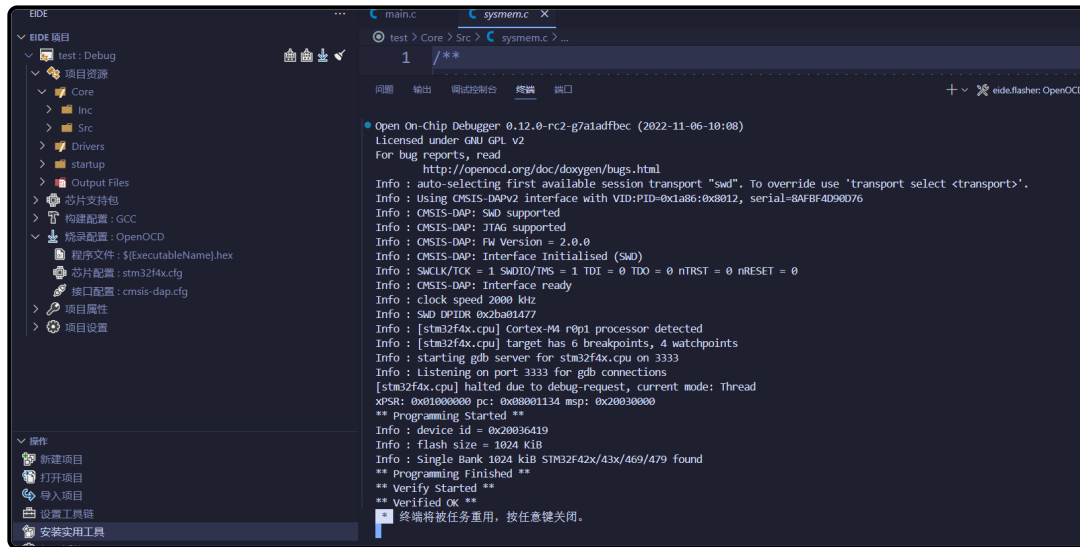
下载完成后,我们可以看到 C:\Users\zuolan\.eide\tools\openocd_7a1adfbec_mingw32\share\openocd\scripts\board 下有很多板子的配置文件,我们找到对应的板子,比如我用的是f429,那么就是stm32f4x.cfg,不同的以此类推,这里都列出来了,我们其实不需要找到文件夹,直接修改就行



我们可以找到这个路径下 `C:\Users\zuolan\.eide\tools\openocd_7a1adfbec_mingw32\share\openocd\scripts\interface`，有很多的探针文件，同理我们也直接在这里搜索就行



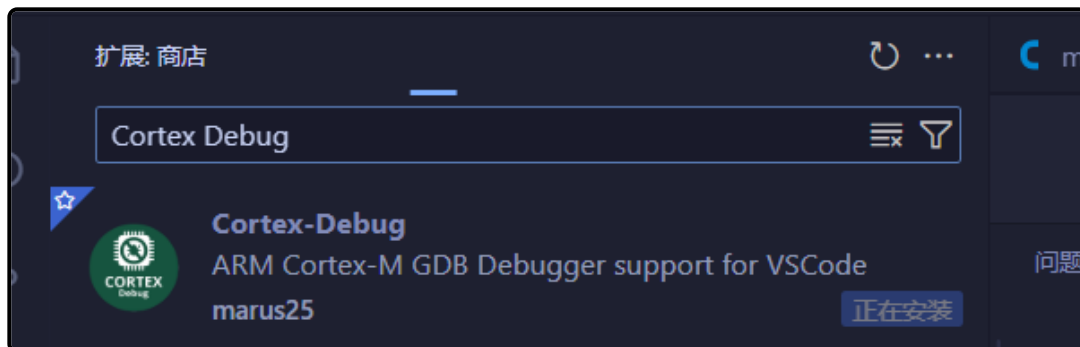
我们根据自己的情况选择是cmsis-dap还是cmsis-dapv2（在左侧直接用cmsis-dap烧录，失败就换就行了），这样就代表烧录成功了



☺ Cortex Debug（调试）

环境配置

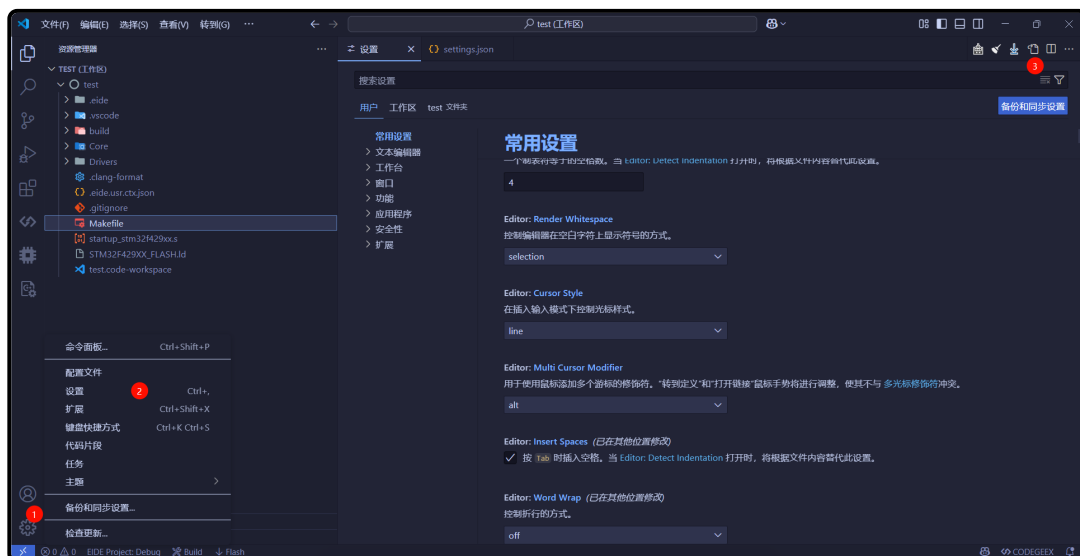
对于调试，我们需要使用Cortex Debug这个插件，安装第一个就行



注意，这个地方必须执行一下我前面的那个EIDE的其他配置，也就是将axf转换为elf

并且必须下载openocd，在daplink的烧录有说

然后我们去json配置一下相关的编译器路径 `F:\path\c++\arm-none-eabi-gcc\bin` 和 openocd 的路径 `C:\Users\zuolan\.eide\tools\openocd_7a1adfbec_mingw32\bin\openocd.exe`



并且注意一下，这里还是要用 / 而不是 \

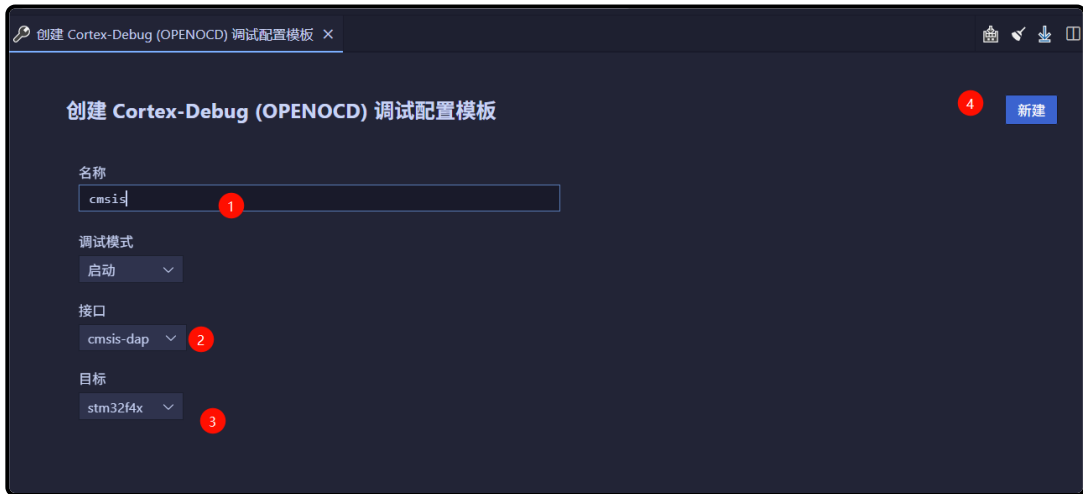


调试配置

我们在build完成工程后，对eide的项目右键，直接选择生成调试器模板，这里面我们选择openocd

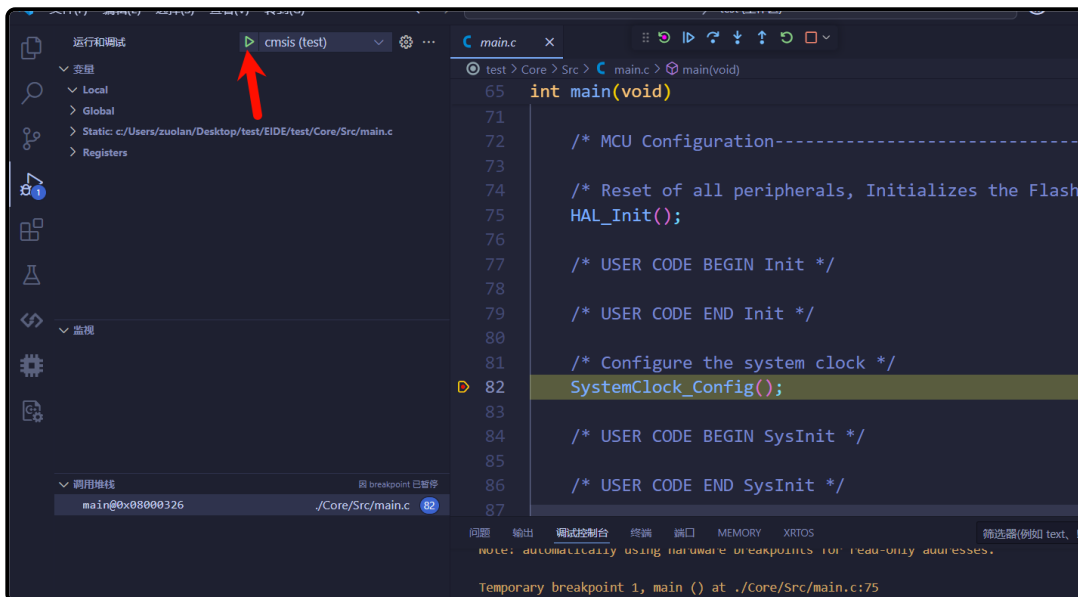


你用啥debug就选啥，我这里改了名字为cmsis，然后直接点击右上角新建就会帮我们生成lanuch（这里cmsis和stlink一样的，但是stlink有很多版本，自己试试哪个版本能跑就行）



```
1 {
2     "version": "0.2.0",
3     "configurations": [
4         {
5             "cwd": "${workspaceRoot}",
6             "type": "cortex-debug",
7             "request": "launch",
8             "name": "cmsis",
9             "serverType": "openocd",
10            "executable": "build/Debug/test.elf",
11            "runToEntryPoint": "main",
12            "configFiles": [
13                "interface/cmsis-dap.cfg",
14                "target/stm32f4x.cfg"
15            ],
16            "toolchainPrefix": "arm-none-eabi"
17        }
18    ]
19 }
20 }
```

确保链接正常就可以直接debug了，相关的操作和keil一致



选中变量右键就可以添加到监视

